

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук

Жуков Данила Дмитриевич

MACHINE LEARNING METHODS IN NETWORK ATTACKS DETECTION

Выпускная квалификационная работа - МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ по
направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Машинное обучение и высоконагруженные системы»

Рецензент:

Научный руководитель:

Правдивец Николай Александрович

преподаватель, ФЭН НИУ ВШЭ

Саночкин Юрий Ильич

старший преподаватель, ФКН НИУ ВШЭ

Москва 2026

Abstract

This thesis contains 4 chapters, 54 pages, 9 figures, 16 tables, 32 references.

This work presents an end-to-end flow-level near real-time network intrusion detection system (IDS) that combines supervised machine learning classification with rule-based detection. The CSE-CIC-IDS2018 dataset (400+ GB of packet captures, 30,8 million network flows) is processed using Zeek to extract 12 baseline features from conn.log, augmented with 5 temporal features computed over a 60-second sliding window per source IP address. Temporal features improve XGBoost macro F1-score from 0,8613 to 0,9605 (+9,9 percentage points) and enable detection of previously unseen attack instances within known behavioral classes.

A two-layer hybrid architecture (temporal ML classifier + rule engine) detects all 7 tested attack types in the controlled lab testbed, compared to 1 out of 7 for the baseline ML model alone. A per-flow autoencoder anomaly detector achieves TPR = 0,01%, indicating that per-flow conn.log features lack the contextual signal required for reliable unsupervised separation in this experimental setting. The system is deployed both as a bare-metal and as a containerized streaming prototype (Zeek -> Kafka -> Spark Structured Streaming), sustaining 25 000 - 33 000 flows per second on bare metal and 24 000 - 30 000 in Docker.

Keywords: network intrusion detection, machine learning, intrusion detection system, XGBoost, Zeek, Apache Kafka, streaming pipeline, sliding time window, CSE-CIC-IDS2018, Docker.

Аннотация

Работа содержит 4 главы, 54 страницы, 9 рисунков, 16 таблиц, 32 использованных источника.

В данной выпускной квалификационной работе представлена система обнаружения сетевых атак на уровне сетевых потоковых соединений в режиме, близком к реальному времени, объединяющая классификацию методами машинного обучения с модулем обработки правил. Обработка датасета CSE-CIC-IDS2018 (400+ ГБ пакетных данных, 30,8 миллиона сетевых потоков) выполнена с помощью Zeek для извлечения 12 базовых признаков из Zeek conn.log, дополненных 5 временными признаками (используется скользящее окно шириной 60 секунд по IP-адресу источника). Временные признаки повышают макро F1 модели XGBoost с 0,8613 до 0,9605 (+9,9 п.п.), обеспечивая обнаружение типов атак, отсутствовавших в обучающей выборке, за счёт поведенческого сходства.

Двухуровневая гибридная архитектура (ML + модуль обработки правил) обнаруживает все 7 протестированных типов атак на лабораторном стенде, тогда как базовая ML-модель обнаруживает лишь 1 из 7. Автоэнкодер на потоковых признаках показывает TPR = 0,01%, что указывает на недостаточность потоковых признаков без временного контекста для устойчивого unsupervised-обнаружения в данной постановке задачи. Система была реализована как в конфигурации на физических серверах, так и в виде контейнеризованного прототипа (Zeek -> Kafka -> Spark Structured Streaming) с пропускной способностью 25 000 - 33 000 потоков/сек на физическом сервере и 24 000 - 30 000 в контейнере Docker.

TABLE OF CONTENTS

Abstract.....	2
Аннотация.....	2
Introduction.....	5
Chapter 1: Literature Review.....	8
1.1 Intrusion Detection Systems	8
1.2 Benchmark Datasets for Network Intrusion Detection.....	9
1.3 Machine Learning Approaches for Network Intrusion Detection	10
1.4 Feature Engineering for Network Traffic Analysis.....	11
1.5 Real-Time Streaming Architectures for Network Security.....	12
1.6 Temporal Feature Engineering for Network Traffic	13
1.7 Summary and Research Gaps	14
Chapter 2: Dataset and Feature Engineering	16
2.1 Network attack categories covered in this work.....	16
2.2 CSE-CIC-IDS2018 Dataset	17
2.3 PCAP Processing with Zeek	18
2.4 Flow Labeling Methodology	19
2.5 Class Distribution	21
2.6 Baseline Feature Set	22
2.7 Temporal Feature Engineering.....	24
2.8 Feature Engineering Design Decisions.....	25
Chapter 3: Model Training and Evaluation	27
3.1 Model Selection	27
3.2 Experimental Setup.....	28

3.3 Baseline Results (12 Features).....	29
3.4 Temporal Results (17 Features)	31
3.5 Autoencoder Experiment (Negative Result)	32
3.6 Ensemble Methods Consideration	33
3.7 Discussion	34
Chapter 4: Real-Time System Design and Live Evaluation	35
4.1 Architecture Overview	35
4.2 Kafka Streaming Transport.....	36
4.3 Two-Layer Hybrid Detection.....	36
4.4 Consumer Implementation.....	37
4.5 Lab Testbed.....	38
4.6 Live Testing: Baseline ML Only.....	41
4.7 Live Testing: Temporal Model with Rule Engine.....	42
4.8 Key findings from live testing	43
4.9 Docker Deployment.....	45
4.10 Performance	46
4.11 Limitations	48
Conclusion	50
References.....	52

Introduction

Rapid growth of internet-connected devices and the escalating sophistication of cyber threats have made automated network security monitoring a critical operational requirement for modern organizations. Intrusion Detection Systems (IDS) serve as a fundamental layer of network defense, monitoring traffic for evidence of unauthorized access or malicious behavior and alerting security personnel in real time. Rule-based systems such as Snort [4] and Suricata [5] reliably detect known attack signatures but are inherently reactive - they cannot identify zero-day exploits or new attack variants. Machine learning offers an alternative: classifiers trained on labeled historical traffic can generalize beyond fixed signatures and adapt to new attack patterns without manual rule authoring.

Despite a large body of published work on machine learning for network intrusion detection, several fundamental problems limit the practical applicability of existing results. First, the dominant feature extraction tool - CICFlowMeter - contains multiple documented implementation issues, including double-counting of the first packet in Packet Length statistics, premature TCP termination on a single FIN packet (rather than bidirectional FIN exchange) - which Engelen et al. [11] identify in CICIDS2017 as the cause of TCP appendices accounting for 25,9% of flow records, and additional artifacts such as NaN/Inf in Flow Bytes/s and Flow Packets/s features and a flowTimeout mismatch between code (120 s) and documentation (600 s) [1, 32]. Because CICIDS2018 datasets were also generated using CICFlowMeter, similar classes of artifacts may be present, although this thesis does not independently quantify their prevalence in CICIDS2018. Getman et al. [1] report that a Random Forest classifier trained on a CICIDS2017 web-attack subset achieves $F1 = 0,971$ on a validation split but drops to $F1 = 0,064$ on traffic from a real network, attributing the degradation to dataset and environment mismatch and arguing for verifying public datasets against the target deployment environment. Second, CICFlowMeter and similar batch tools compute features only after a flow has fully terminated. In a streaming inference scenario, a TCP connection in progress cannot be represented by the same feature vector used for training, since bidirectional statistics are undefined before flow closure. Third, to my knowledge the path from a trained model to a production real-time system is almost never described in the literature: Kafka consumer throughput, model serialization, and feature consistency between training and inference are treated as implementation details rather than research contributions.

This thesis addresses these problems simultaneously. We use Zeek conn.log as the exclusive feature source, bypassing CICFlowMeter entirely. Zeek is a production-grade, open-source network analysis framework widely used in research and operational

security environments. It generates a conn.log record in the moment when connection closes - the same timing and feature set that would be available in a streaming inference scenario. The zeek-kafka plugin allows Zeek to publish conn.log records to Apache Kafka in real time, providing a natural integration point for a streaming classification pipeline where the same feature extractor operates in both training and inference contexts.

As the primary dataset we use CSE-CIC-IDS2018 [10] - one of the most comprehensive publicly available network traffic benchmarks, comprising 400+ GB of raw packet captures over 10 days with a diverse attack taxonomy: DoS variants, DDoS, brute-force, infiltration, and a botnet. Processing, deduplication, and labeling of this data produces 30,8 million labeled network flows. Random Forest, LightGBM and XGBoost classifiers trained on 12 Zeek conn.log features achieve weighted $F1 = 0,9947$, competitive with published baselines that use 80 CICFlowMeter features.

This work targets flow-level near real-time intrusion detection. Classification is performed on a per-flow basis in the moment Zeek emits the corresponding conn.log record - that is, when the underlying TCP/UDP connection closes (or when Zeek's inactivity timeout expires). End-to-end latency from the first packet of a short-lived attack to the first alert is typically 3–5 seconds. Strict packet-level real-time detection (sub-millisecond, in-line traffic decisions) is therefore out of scope. The term real-time is used throughout this thesis in this flow-level near real-time sense.

The contributions of this thesis are:

1. Reproducible Zeek-based PCAP processing pipeline for CSE-CIC-IDS2018, including an automated fix for truncated UCAP captures, producing 30,8 million labeled network flows across 9 classes, including BENIGN and 8 attack classes.
2. Temporal sliding-window feature engineering: five per-source-IP aggregate statistics, computed over a 60-second sliding-window that improve XGBoost macro $F1$ from 0,8613 to 0,9605 (+9,9 percentage points) and enable generalization to attack types absent from the training data.
3. Two-layer hybrid detection architecture combining a temporal XGBoost classifier with the rule engine, achieving detection of 7 out of 7 tested attack types in the controlled lab testbed described in §4.5 (compared to 1 out of 7 for the baseline ML model alone), despite both models exceeding weighted $F1 = 0.994$ in offline evaluation. This contrast directly demonstrates that high offline benchmark metrics severely overestimate operational detection capability.

4. Negative result: a per-flow autoencoder anomaly detector achieves $TPR = 0,01\%$ on conn.log features, confirming that per-flow conn.log features are insufficient for anomaly detection without temporal context.
5. Containerized end-to-end streaming prototype (Zeek -> Kafka -> Spark Structured Streaming) with documented Docker deployment, sustaining 25,000–33,000 flows per second on standard modern hardware.

The thesis is organized as follows:

Chapter 1 reviews related work in intrusion detection systems, benchmark datasets, machine learning methods, temporal feature engineering, and streaming architectures.

Chapter 2 describes the CSE-CIC-IDS2018 dataset, the Zeek-based processing pipeline, flow labeling methodology, and the design of both baseline and temporal feature sets.

Chapter 3 presents model training and offline evaluation, including baseline models, temporal models, and autoencoder experiment.

Chapter 4 describes the real-time streaming system design, rule engine, and live evaluation against 7 attack types on a lab testbed.

The thesis concludes with a summary of findings, limitations, and directions for future work.

Chapter 1: Literature Review

1.1 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a software or hardware component that monitors network traffic or host activity for evidence of policy violations, unauthorized access, or malicious behavior. The concept was first described in Anderson's 1980 technical report for the U.S. Air Force [2], which proposed automated audit trail analysis as a security control. Denning [3] subsequently formalized an anomaly-based detection model based on statistical profiles of user behavior, laying the theoretical foundation for statistical IDS research that continues to the present day.

Modern IDSs are classified along two orthogonal dimensions. The first distinguishes Host-based IDS (HIDS) from Network-based IDS (NIDS): HIDS analyzes system calls, file integrity, and process behavior on individual hosts, while NIDS passively observes network traffic at important points, such as router interfaces or core switch span ports. This thesis focuses exclusively on NIDS, since network-level monitoring can protect heterogeneous device fleets - including IoT devices, embedded systems, and legacy hardware - without requiring agents installed on each endpoint.

The second dimension distinguishes signature-based detection from anomaly-based detection. Signature systems, exemplified by Snort [4] and Suricata [5], match traffic against databases of known attack patterns represented as rule sets. They achieve near-zero false positive rates on known threats but cannot detect novel attacks or zero-day exploits. Anomaly systems build a model of normal behavior and flag deviations: they can in principle detect previously unseen attack patterns but are historically prone to high false positive rates when legitimate traffic changes. Machine learning-based anomaly detection, trained on labeled examples of both benign and malicious traffic, occupies a middle ground: it generalizes beyond fixed signatures while benefiting from the supervision signal of labeled data.

Commercial NIDS products have converged on hybrid architectures. Cisco Stealthwatch, Darktrace, and similar platforms combine rule-based detection with behavioral analytics powered by machine learning. The academic literature, however, has focused predominantly on benchmark evaluation of classification algorithms, leaving deployment architecture - including feature extraction pipelines, streaming infrastructure, and real-time latency - as an open engineering problem. This thesis directly addresses that gap.

1.2 Benchmark Datasets for Network Intrusion Detection

The availability of labeled network traffic datasets is a prerequisite for reproducible research on machine learning-based NIDS. The field has progressed through several generations of benchmarks, each addressing limitations of its predecessors.

The KDD Cup 1999 dataset [6] dominated the field for over a decade. Derived from the DARPA 1998 network simulation, it contains approximately 5 million labeled connection records. Tavallaee et al. [7] identified fundamental flaws: 78% of training instances are exact duplicates, two probe attack types account for 98% of test instances, and the network simulation does not reflect realistic traffic patterns. The NSL-KDD dataset [7] removed duplicates but retained simulation artifacts.

UNSW-NB15 [8], created at the University of New South Wales, offered a more realistic simulation using commercial traffic generation tools and labeled nine attack families. While methodologically improved, it remained a controlled simulation. The CICIDS2017 [9] and CSE-CIC-IDS2018 [10] datasets, produced by the Canadian Institute for Cybersecurity, represent the current state of the art: they were generated on large, emulated networks with detailed attack schedules against known target machines, providing both raw PCAP files and pre-computed feature vectors.

CSE-CIC-IDS2018 is the largest and most comprehensive of these datasets. It spans 10 days (February 14 to March 2, 2018) on an AWS-hosted emulated network with 500 Windows machines organized as desktops, file servers, and Ubuntu attack server. Attack types include DoS variants (Hulk, Slowloris, SlowHTTPTest, GoldenEye), DDoS (HOIC, LOIC-UDP), brute-force attacks (FTP, SSH), infiltration, web attacks (SQL injection, XSS brute-force), and a botnet. Raw PCAP files with total size 400+ GB and are available at AWS S3.

A critical limitation of CICFlowMeter-derived CIC datasets is the reliability of extracted flow features. Getman et al. [1] report several CICFlowMeter implementation issues, including double-counting of the first packet in Packet Length statistics, premature TCP session termination on the first FIN packet, duplicated Fwd Header Length, incorrect TCP packet length calculation due to Ethernet padding, and inconsistent Packet Length Mean / Average Packet Size calculations. Engelen et al. [11] independently analyze CICIDS2017 and identify problems in traffic generation, flow construction, feature extraction, and labelling, including TCP appendices caused by premature flow termination that account for 25,9% of CICIDS2017 flow records. Russian-language practitioner summary [32] additionally lists NaN/Inf values in the Flow Bytes/s and Flow Packets/s features and a flowTimeout mismatch between the implementation (120 s) and the tool documentation (600 s). Models trained on CICFlowMeter-extracted

CICIDS2017 features achieve $F1 = 0,971$ on a validation split but drop to $F1 = 0,064$ on traffic from a real network [1]. Getman et al. attribute this degradation to a combination of dataset and environment mismatch. Because CICIDS2018 was also generated using CICFlowMeter, similar classes of artifacts may be present, although this thesis does not independently quantify their prevalence in CICIDS2018. These findings motivate our decision to bypass CICFlowMeter entirely and extract features directly from Zeek conn.log.

1.3 Machine Learning Approaches for Network Intrusion Detection

The application of machine learning to network intrusion detection has a history spanning three decades. Early work in the 1990s explored neural networks and decision trees on the KDD dataset, establishing a tradition of classification-based evaluation that persists today. This section surveys the methodological landscape relevant to this thesis.

Classical supervised learning methods - decision trees, naive Bayes, k-nearest neighbors, and support vector machines - were systematically compared on KDD99 throughout the 2000s. Ensemble methods emerged as consistently stronger performers in the 2010s. Random Forest [12] in particular became a standard baseline due to its resistance to irrelevant features, interpretable feature importances, and competitive accuracy without extensive hyperparameter tuning. Gradient boosting implementations - AdaBoost, XGBoost [13], LightGBM [14] - match or exceed Random Forest performance on most benchmark tasks at lower inference cost.

The CICIDS2017 and CICIDS2018 benchmarks attracted substantial attention from the machine learning community. Leevy and Khoshgoftaar [15] systematically surveyed intrusion detection models based on CSE-CIC-IDS2018 and reported that tree-based ensemble methods such as Random Forest and XGBoost achieved some of the strongest weighted F1-scores (above 0,995) with training times under one hour on consumer hardware. Ferrag et al. [16] conducted a comprehensive survey of deep learning methods on the same dataset: while recurrent neural networks and attention-based architectures improved detection of sequential attack patterns, the performance gains over ensemble methods were marginal for most attack classes while training computational costs increased by orders of magnitude.

Deep learning approaches for NIDS can be grouped into three families. Convolutional neural networks have been applied to both raw packet bytes - treating the payload as a 1D signal - and to flow-level feature matrices represented as images. Recurrent architectures (LSTM, GRU) capture temporal dependencies in sequences of flows from

the same source, which is particularly relevant for botnet and brute-force detection where the attack pattern manifests across multiple connections rather than within a single flow. Autoencoder-based anomaly detectors learn a compressed representation of normal traffic and flag high reconstruction errors, enabling detection of novel patterns without labeled attack data.

A systematic limitation of most published work, identified by Sommer and Paxson [17], is the mismatch between laboratory evaluation and operational deployment. Models trained on one network’s traffic generalize poorly compared to another due to differences in protocol mix, traffic volume, and user behavior. This distribution shift problem is compounded when training data contains label noise or feature corruption. Our approach mitigates distribution shift at the feature level by using Zeek conn.log, which is transport-layer agnostic and produces the same 12-column record regardless of application protocol, traffic volume, or network topology.

1.4 Feature Engineering for Network Traffic Analysis

Feature engineering - the selection and transformation of raw data into machine-readable representations is as consequential for NIDS performance as algorithm selection. Network traffic features can be organized in a hierarchy: packet-level features (header fields, inter-packet timing, payload entropy), flow-level features (aggregate statistics over a unidirectional or bidirectional flow), and behavioral features (temporal patterns across multiple flows from the same source or destination).

Packet-level features offer the richest information but require deep packet inspection (DPI), which raises privacy concerns and is computationally expensive at high line rates. Payload-based features are inapplicable to encrypted traffic (now the majority of internet traffic) without SSL/TLS interception. As a result, the research community has largely shifted toward flow-level features that do not require payload access.

The IETF IP Flow Information Export (IPFIX) protocol standardized vocabulary for flow meters. CICFlowMeter, developed for the CIC datasets, extends IPFIX with 80 bidirectional statistics including inter-arrival time moments, active and idle time intervals, and payload length distributions. As discussed in Section 1.2, this richness comes at the cost of reproducibility due to implementation bugs.

Zeek (formerly Bro) takes a fundamentally different approach. Rather than computing flow statistics, Zeek runs a scripting language that reconstructs protocol state machines at multiple layers simultaneously. The conn.log output records connection-level metadata including duration, byte and packet counts in both directions, connection state

(established, rejected, half-open), transport protocol, application-layer service identified by DPI, and a history string encoding the sequence of TCP flags and data packets observed. This record is produced when the connection closes - at the same point in time as the flow record that a streaming classifier would receive. Feature availability at closure time is the key property ensuring train-inference consistency.

CICFlowMeter’s 80-feature representation contains many aggregate statistics based on complete bidirectional flows, packet inter-arrival times, active/idle periods, and packet length distributions. Reproducing such features in a streaming Zeek-based pipeline would require maintaining additional packet-level state beyond conn.log and would weaken train-inference consistency. Gustavsson [18] explores an alternative Zeek-based feature extraction approach for CICIDS2017, showing that Zeek logs can be used as a practical basis for ML-based NIDS experiments. The selected Zeek conn.log fields, by contrast, consist of bounded per-connection counters and state fields written once when the connection closes. This provides a practical justification for the feature selection.

Recent work has explored neural feature learning directly from raw PCAP data. ET-BERT [19] and TrafficFormer [20] apply transformer architectures to byte sequences of network packets, learning representations without manual feature engineering. While these approaches show promise on encrypted traffic classification, they require GPU acceleration for real-time processing and cannot directly consume Zeek structured log output. Integrating such approaches into a Kafka-based pipeline is an open research question we leave for future work.

1.5 Real-Time Streaming Architectures for Network Security

Deploying ML-based intrusion detection in a production environment requires an architecture that classifies network events with latency measured in seconds or less. Batch-processing pipelines - collect traffic, extract features, classify, alert - introduce minutes-to-hours latency, which is incompatible with the requirement of detecting ongoing attacks before significant damage occurs.

Apache Kafka [21] has become the standard for high-throughput event streaming in enterprise environments. Its log-structured commit log, partitioned topics, and consumer group semantics sustain millions of events per second with durability guarantees. Network monitoring use case maps naturally to Kafka: the sensor (Zeek) acts as a producer, Kafka provides transport and buffering, and one or more consumers apply detection logic and route alerts. The separation of capture (Zeek), transport (Kafka), and

classification (Spark consumer) allows each component to scale independently and fail without affecting the others.

Apache Flink [22] and Apache Spark Streaming [23] provide higher-level abstractions for stateful stream processing, including windowed aggregations, joins, and pattern matching. They are appropriate when detection logic requires temporal context across multiple events - for example, computing the connection rate from a source IP over a 60-second window. Their complexity and resource requirements, however, exceed what is necessary for a per-flow classification baseline. The streaming prototype in this thesis uses a Spark Structured Streaming consumer with XGBoost inference. Temporal aggregation over sliding windows is described in Chapter 2 and Chapter 4.

Several prior studies apply ML classification to network intrusion detection in different operational regimes. Ioannou and Vassiliou [24] applied SVM-based classification to IoT network traffic, evaluating C-SVM and One-Class SVM for detecting attacks in constrained environments and reporting up to 100% accuracy on known network topologies. Ahmad et al. [25] compared SVM, Random Forest, and Extreme Learning Machine on the NSL-KDD dataset, reporting different performance trade-offs among classical ML classifiers for intrusion detection. Gnanavarothayan et al. [27] developed a real-time threat detection prototype based on packet-level network analysis, demonstrating end-to-end alerting with sub-second latency on a constrained testbed. Zubkov [26] proposed a microservice architecture for streaming network traffic events, focusing on the engineering trade-offs of decoupling capture, transport and analytics layers. None of these studies, however, addressed the feature-consistency problem between batch training and streaming inference end-to-end. Zeek-kafka plugin - used at several national research networks and originally developed as part of the Apache Metron project - provides a production-tested integration point that bridges this gap by publishing the same conn.log record format in both offline and streaming contexts.

1.6 Temporal Feature Engineering for Network Traffic

Flow-level features describe individual connections in isolation, but many network attacks manifest as patterns across multiple connections. Temporal feature engineering - computing aggregate statistics over sliding time windows per source or destination IP - has been shown to improve detection of attacks whose per-flow signatures are indistinguishable from benign traffic.

The concept of multi-resolution aggregation is identified as one of four key research directions for modern NIDS by recent surveys on deep learning for contextualized

network flow analysis [28]. This approach combines local features (per-flow) with global views (per-source-IP aggregates over time), capturing both the properties of individual connections and the behavioral context in which they occur.

Hybrid IDS architectures that combine sliding-window statistics with supervised classifiers have demonstrated improved detection of volumetric attacks. A recent study [29] proposed a hybrid IDS using sliding-window information entropy features alongside gradient-boosted classifiers, reporting improved detection of DDoS attacks whose individual flows are statistically normal. Similarly, research on NIDS with sliding time windows [30] demonstrated that per-source-IP connection rate and byte volume aggregates over 30–120 second windows significantly improve F1-scores for DoS and brute-force attacks on CICIDS datasets.

Graph Neural Networks represent an emerging approach that captures relational structure between hosts. Zhong et al. [31] survey GNN-based IDS methods, noting their effectiveness for coordinated attacks such as botnets where command-and-control communication structure is a strong signal. However, GNN-based approaches operate in offline batch mode and lack streaming inference implementations suitable for real-time deployment.

Transformer architectures applied to traffic representations, such as TrafficFormer [20], show promising offline results but require GPU acceleration and a fundamentally different input representation - sequences of N consecutive flows rather than individual flow records. No streaming implementation exists at the time of writing.

The temporal sliding-window approach adopted in this thesis occupies a practical middle ground: it captures cross-flow behavioral patterns with bounded per-source-IP state and can be optimized to $O(1)$ updates via running counters, although the current prototype uses an $O(k)$ per-source-IP window update as discussed in Section 2.7. It operates within the existing Kafka streaming pipeline and requires no GPU resources. The five temporal features (connection rate, unique destination ports, mean duration, standard deviation of duration, and bytes per second) are computed per source IP over a 60-second window, enabling detection of attacks whose per-flow features are indistinguishable from benign traffic.

1.7 Summary and Research Gaps

This review identifies four specific gaps that this thesis addresses:

Gap 1 - Feature reliability. Published studies using CICFlowMeter features, report inflated performance metrics due to systematic feature corruption. No prior study has evaluated classification performance on CSE-CIC-IDS2018 using Zeek conn.log features exclusively.

Gap 2 - Train-inference consistency. The feature extraction tool used for training is rarely the same as the tool used during inference. To the best of our knowledge, no prior study integrates real-time traffic capture, streaming transport, and ML classification into a single end-to-end pipeline where the same feature extractor operates in both contexts.

Gap 3 - End-to-end deployment with live evaluation. Most published work treats classification accuracy as the sole evaluation criterion, evaluating only on offline test sets. Deployment architecture - streaming latency, model serving, alert routing - and live testing against actual attack traffic are rarely addressed. This thesis contributes both a documented streaming prototype and a live evaluation on seven attack types demonstrating the gap between offline metrics and operational detection capability.

Gap 4 - Temporal features for flow-level IDS. While temporal aggregation is recognized as important in literature, no prior study on CICIDS2018 has combined per-source-IP sliding-window features with Zeek conn.log and evaluated the improvement both offline (macro F1) and live detection rate.

Table 1. Positioning of this thesis relative to prior studies on CICIDS datasets

Study	Dataset	Features	Real-time	Train=Infer	Wt. F1
Leevy et al. 2020 [15]	CIC-2018	CICFlowMeter	No	No	0,995
Ferrag et al. 2020 [16]	CIC-2017	CICFlowMeter	No	No	0,994
Ahmad et al. 2018 [25]	NSL-KDD	NetFlow	No	No	0,971
Ioannou 2021 [24]	Custom IoT	SVM features	No	No	0,990
This work	CIC-2018	Zeek conn.log	Yes	Yes	0,9947

As shown in Table 1, this thesis is unique in combining Zeek conn.log (rather than CICFlowMeter or NetFlow) as the feature source for both training and real-time inference on the CSE-CIC-IDS2018 dataset, and in deploying a Kafka-based streaming pipeline with a consistent feature extractor.

Chapter 2: Dataset and Feature Engineering

2.1 Network attack categories covered in this work

Network-based attacks observed in the wild span a wide spectrum, from low-bandwidth reconnaissance to high-volume distributed denials of service. The present work focuses on seven attack types reproduced in the live evaluation reported in Chapter 4 - six are present in the CSE-CIC-IDS2018 dataset (two volumetric HTTP-layer denial-of-service variants, two slow-rate HTTP attacks and two credential brute-force attacks) and one is an additional unseen TCP-layer SYN flood scenario used to evaluate generalization within the volumetric DoS family. This section provides the background concepts required to interpret the rest of the thesis.

Denial-of-service. A denial-of-service (DoS) attack aims to exhaust a finite resource of a server - bandwidth, CPU, memory, file descriptors, worker threads or connection slots - so that legitimate clients can no longer be served. When the attack traffic is generated by a single source it is called a DoS, when it originates from a coordinated set of sources it is called a distributed denial-of-service (DDoS). The two broad strategies are volumetric and slow-rate: volumetric attacks overwhelm the target with as much traffic as possible per second, while slow-rate attacks send traffic at a deliberately low rate but in a way that holds finite resources open indefinitely. The two strategies leave very different signatures on flow-level data, and a single-feature detector that handles one of them well typically fails on the other.

SYN flood. The SYN flood is a classical transport-layer (L4) volumetric attack. The attacker sends a high rate of TCP SYN segments to the target, often with spoofed source addresses, but never completes the three-way handshake. Each half-open connection consumes a slot in the server's connection backlog. Once the backlog is full, legitimate connections are refused. On flow-level data the attack manifests as an unusually high arrival rate of SYN-only or REJ connections from one or a few sources to a single destination port.

Volumetric HTTP denial-of-service. HTTP floods such as DoS-Hulk and DoS-GoldenEye operate at the application layer (L7). They issue a high rate of HTTP GET or POST requests to a web server, with payloads engineered to defeat caches and signature-based defences - randomised URI parameters and User-Agent strings in Hulk, KeepAlive and Cache-Control: no-cache headers in GoldenEye. The traffic looks superficially like ordinary web browsing, but the request rate per source is one to two orders of magnitude higher than benign behaviour, which makes statistical features such as connection rate and bytes-per-second discriminative. The DDoS-HOIC class present

in the CIC-IDS2018 label set falls in the same family but assumes a coordinated multi-host attacker.

Slow-rate HTTP denial-of-service. DoS-Slowloris and DoS-SlowHTTPTest invert the volumetric strategy: instead of overwhelming throughput, they open a large number of HTTP connections and keep each one alive by sending tiny, infrequent fragments of the request - one header line every hundred seconds in the case of Slowloris, slow-read or slow-body fragments in SlowHTTPTest. The attack succeeds against thread-per-connection servers because each held-open connection consumes a worker thread, with no need for high bandwidth. On the wire the traffic is low-volume and indistinguishable from benign HTTP on classical packet-rate metrics, but the connections are unusually long-lived for HTTP and carry almost no bytes per second.

Credential brute-force. SSH-BruteForce and FTP-BruteForce target authentication services rather than service availability. The attacker iterates through a credential dictionary, attempting many username/password combinations against the target service (TCP/22 for SSH, TCP/21 for FTP). Each attempt typically runs over its own short TCP connection that is closed by the server after a fixed number of failed credentials. The dominant network-level signature is therefore a rapid burst of short connections from one source to a single destination port, which can be matched by simple flow-level counting rules.

The six dataset attack types reproduced in the lab, together with the additional SYN flood scenario, cover the live-evaluation behavioral families. The offline dataset additionally includes Bot and DDoS-HOIC. Bot is not reproduced live because it requires multi-host command-and-control behavior, and DDoS-HOIC requires multiple coordinated booster hosts beyond the lab topology.

2.2 CSE-CIC-IDS2018 Dataset

CSE-CIC-IDS2018 is a network intrusion detection benchmark produced by the Canadian Institute for Cybersecurity at the University of New Brunswick [10]. It was generated on an AWS-hosted emulated enterprise network comprising approximately 500 machines organized into a victim infrastructure (subnet 172.31.69.0/24, Ubuntu servers and 172.31.64.0/24, Windows desktops) and a dedicated attack infrastructure (subnet 172.31.70.0/24). The capture spans 10 business days from February 14 to March 2, 2018.

The dataset covers seven categories of attacks executed against the victim machines according to a published schedule: DoS variants (Hulk, Slowloris, SlowHTTPTest,

GoldenEye), DDoS (HOIC, LOIC-UDP), brute-force attacks targeting FTP and SSH services, web application attacks (SQL injection, XSS, brute-force login), infiltration, and a botnet campaign using the Zeus/Ares toolkit. Raw packet captures total approximately 400+ GB and are distributed as per-machine PCAP files via AWS S3.

The dataset is distributed in two forms: raw PCAP files and pre-computed marked CSV feature vectors extracted by CICFlowMeter. As discussed in Section 1.2, CICFlowMeter contains documented implementation issues that may produce inconsistent or unreliable feature values. In this work, the raw PCAP files serve as the sole data source, CICFlowMeter outputs are not used at any stage.

2.3 PCAP Processing with Zeek

All PCAP files are processed using Zeek 7.0.4, an open-source network analysis framework widely deployed at national research networks and security operations centers. Zeek reconstructs transport-layer state machines from raw packets and produces structured logs, of which `conn.log` - a per-connection summary record - is the primary output used in this work.

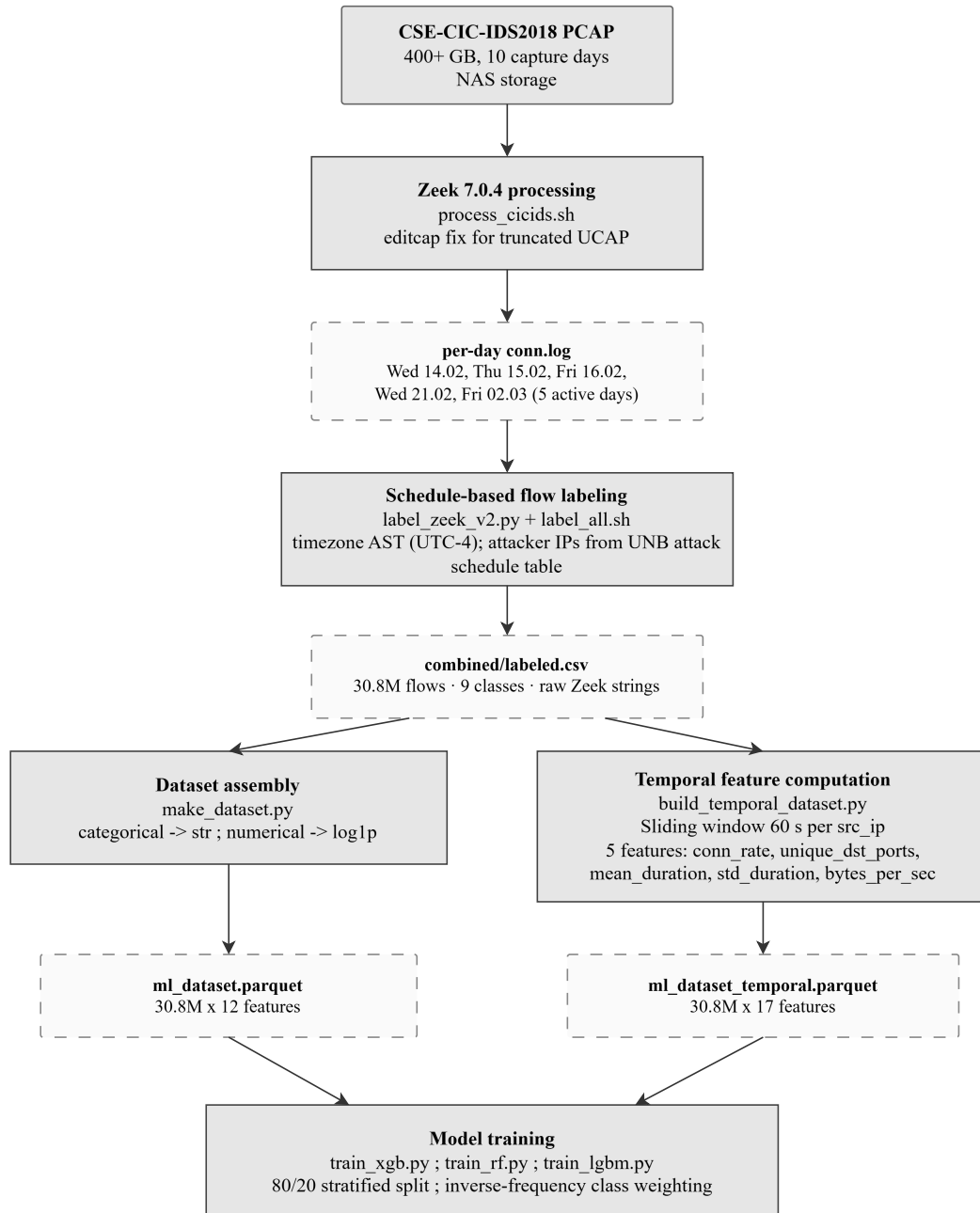
Figure 1 illustrates the end-to-end PCAP processing pipeline, from raw captures on the NAS to trained models in the baseline and temporal configurations.

The processing pipeline operates as follows: for each of the 10 capture days, the pipeline iterates over all per-machine PCAP files and invokes Zeek with checksum validation disabled (`zeek -C -r <file>`). The PCAP files fall into three categories by naming convention: DCAP files (Windows desktops, subnet 172.31.64.x), capWIN files (Windows workstations, subnet 172.31.67.x), and UCAP files (Ubuntu victim servers, subnet 172.31.69.x). A dataset-specific issue affects the UCAP files: they are truncated at the final packet, causing Zeek to terminate with a fatal read error. The pipeline handles this automatically by applying `editcap` (from the Wireshark toolkit) to produce a repaired copy with all valid packets preserved, then re-running Zeek on the repaired file. Per-day `conn.log` outputs are concatenated and sorted by timestamp to produce a single unified log.

The decision to process raw PCAPs through Zeek rather than using the distributed CSV files addresses two problems simultaneously. First, it avoids the feature reliability issues introduced by CICFlowMeter. Second, it ensures that the feature extraction process during training is identical to the one used during real-time inference: in both cases, Zeek produces the same `conn.log` record format with the same fields, computed by the

same code. This property is a prerequisite for reliable deployment of the trained model on live traffic.

Figure 1 - the processing pipeline:



2.4 Flow Labeling Methodology

Zeek conn.log records do not carry attack labels. To produce a labeled dataset, we implement a schedule-based labeling procedure that assigns attack labels to flows based on the published attack timetable and the IP addresses of attacker and victim machines [10].

The CSE-CIC-IDS2018 dataset spans ten capture days, each scripted to generate a specific family of attacks alongside benign background traffic. Table 2 lists the official schedule together with the inclusion decision applied in this work. Of the ten days, four contribute attack labels in full (days 1, 2, 3, 10), one contributes partially (day 5), and five are excluded from attack labeling (days 4, 6, 7, 8, 9). Day 4 is excluded because both attack classes scheduled for that day fall outside the scope of this work.

Table 2. CIC-IDS2018 attack schedule and inclusion status in this work

Day	Date (2018)	Attack types in CIC-IDS2018	Status	Inclusion rationale
1	Wed, 14 Feb	FTP-BruteForce, SSH-BruteForce	Included	Burst of short connections to TCP/21 and TCP/22, clear flow-level signature
2	Thu, 15 Feb	DoS-GoldenEye, DoS-Slowloris	Included	HTTP-layer DoS visible as anomalous connection patterns to TCP/80
3	Fri, 16 Feb	DoS-Hulk, DoS-SlowHTTPTest	Included	Volumetric and slow HTTP attacks visible in conn.log
4	Tue, 20 Feb	DDoS-LOIC-UDP, DDoS-LOIC-HTTP	Excluded	LOIC-UDP yields only ~120 flows (insufficient for stratified training), LOIC-HTTP signature overlaps with HOIC on day 5 and is excluded for class clarity
5	Wed, 21 Feb	DDoS-LOIC-HTTP, DDoS-HOIC	Partial	DDoS-HOIC retained as an independent class, DDoS-LOIC-HTTP excluded (see day 4)
6	Thu, 22 Feb	Brute Force-Web, Brute Force-XSS, SQL Injection	Excluded	Application-layer (L7) attacks, no distinguishable L3/L4 signature in conn.log
7	Fri, 23 Feb	Brute Force-Web, Brute Force-XSS, SQL Injection	Excluded	Same as day 6

8	Wed, 28 Feb	Infiltration	Excluded	Multi-stage attack, detection requires application-layer or endpoint correlation that conn.log fields cannot supply
9	Thu, 1 Mar	Infiltration	Excluded	Same as day 8
10	Fri, 2 Mar	Botnet (Zeus / Ares C2)	Included	Included as a separate dataset class, detection remains difficult with conn.log-only features

The exclusion of application-layer attacks (days 6–9) is a deliberate scope choice: conn.log records describe TCP/UDP connections at the network and transport layers, and an attack that produces no distinguishable connection-level signature cannot be detected by a model trained on conn.log features alone. Benign background traffic from excluded days is still present in the labelled dataset under the BENIGN label. This boundary is acknowledged in 4.11 (Limitations).

The labeling script reads the official attack schedule spreadsheet, which specifies for each attack: the date, start and end times, attacker IP addresses, victim IP addresses, and attack type. All timestamps in the schedule use Atlantic Standard Time (UTC–4), matching the University of New Brunswick’s time zone. For each conn.log record, the script checks whether the flow time interval overlaps with any attack window and whether the flow involves a known attacker-victim IP pair in either direction. A flow is labeled with the corresponding attack type if both conditions are met, otherwise, it is labeled BENIGN.

The final labeled dataset contains 30,8 million flows across 9 classes.

2.5 Class Distribution

Table 3 presents the class distribution of the labeled dataset. The distribution exhibits severe class imbalance: BENIGN traffic accounts for 89,1% of all flows, while the smallest attack class (DoS-Slowloris) contains fewer than 11 000 flows.

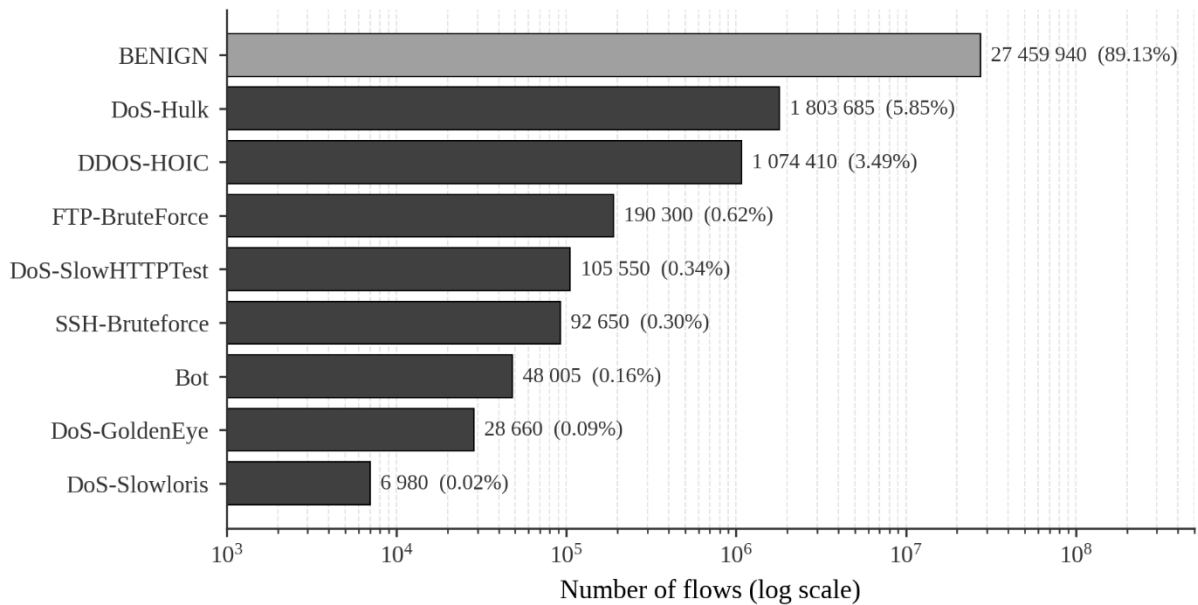
Table 3. Class distribution of the labeled dataset (30,8M flows)

Class	Flows	Percentage
BENIGN	27 459 940	89,13%
DoS-Hulk	1 803 685	5,85%
DDOS-HOIC	1 074 410	3,49%
FTP-BruteForce	190 300	0,62%

DoS-SlowHTTPTest	105 550	0,34%
SSH-BruteForce	92 650	0,30%
Bot	48 005	0,16%
DoS-GoldenEye	28 660	0,09%
DoS-Slowloris	6 980	0,02%
Total	30 810 180	100%

This imbalance reflects realistic network conditions, where benign traffic vastly outnumbers attack traffic. To prevent the classifiers from ignoring rare attack classes and over-optimizing for the BENIGN majority class, inverse-frequency class weights are used during training. Under this scheme, errors on rare classes receive higher weight in the loss function, while errors on frequent classes receive lower weight.

Figure 2 visualizes the imbalance on a logarithmic scale: BENIGN traffic exceeds the smallest attack class (DoS-Slowloris) by more than three orders of magnitude:



2.6 Baseline Feature Set

Twelve features are extracted from each Zeek conn.log record. These features are selected based on three criteria:

1. Availability at connection closure time, ensuring the same values are computable during both offline training and real-time inference.
2. Independence from payload content, allowing the system to operate on encrypted traffic.

3. Absence of unbounded packet-level buffering, ensuring bounded per-connection state.

Table 4. Baseline feature set (12 features)

Feature	Type	Description
duration	Numeric	Connection duration in seconds
orig_bytes	Numeric	Bytes sent by originator
resp_bytes	Numeric	Bytes sent by responder
orig_pkts	Numeric	Packets sent by originator
resp_pkts	Numeric	Packets sent by responder
orig_ip_bytes	Numeric	IP-level bytes (originator, including headers)
resp_ip_bytes	Numeric	IP-level bytes (responder, including headers)
proto	Categorical	Transport protocol (tcp, udp, icmp)
conn_state	Categorical	Connection state (SF, S0, REJ, RSTO, etc.)
history	Categorical	Sequence of TCP flags and data events observed
service	Categorical	Application-layer protocol identified by Zeek DPI
id.resp_p	Integer	Destination port number

Source and destination IP addresses (`id.orig_h`, `id.resp_h`) are excluded from the feature set to prevent data leakage and memorization of dataset-specific host identities. In the CICIDS2018 dataset, attack scenarios are tied to fixed IP address assignments. Including these features would allow the model to learn a direct mapping between IP addresses and attack labels rather than generalized traffic patterns.

This issue is particularly critical under a random train-test split, where the same IP addresses appear in both training and test sets, leading to artificially inflated performance.

The source IP address is used exclusively for temporal feature aggregation (Section 2.7) and is not provided to the classifier.

The seven numeric features undergo a $\log(1+x)$ transformation to reduce the dynamic range of byte and packet counts, which span six orders of magnitude. The four categorical features are encoded using per-feature `LabelEncoder` instances fitted on the training split. The destination port is used as an integer without transformation.

2.7 Temporal Feature Engineering

The baseline feature set describes each connection in isolation. However, many network attacks manifest as patterns across multiple connections from the same source: DoS attacks produce hundreds/thousands of connections per second, brute-force attacks generate repeated authentication attempts, and botnets exhibit periodic beaconing to command-and-control servers (botnets were excluded from lab tests). To capture these cross-flow patterns, we compute five temporal features using a 60-second sliding window aggregated per source IP address.

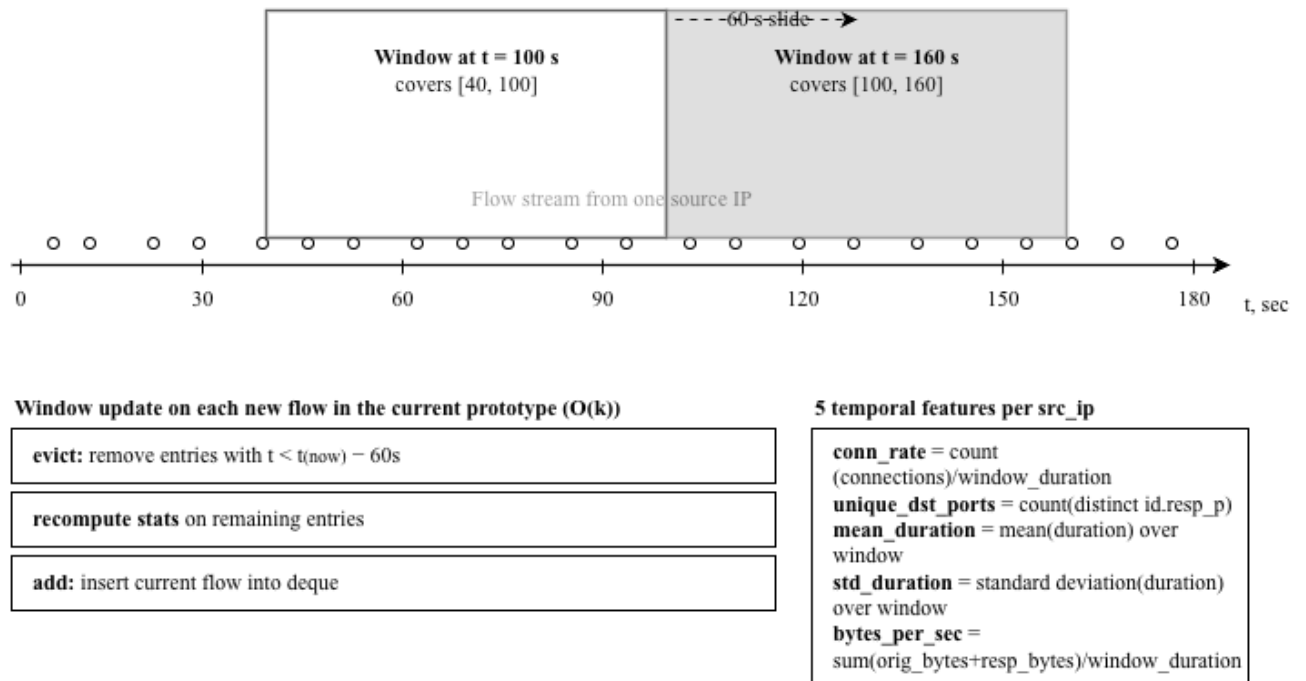
Table 5. Temporal features (5 additional features, 60-second sliding window per source IP)

Feature	Formula	Attack signal
conn_rate	Connections/window_duration	DoS/DDoS: 500–1000 conn/s vs. BENIGN: 1–5 conn/s
unique_dst_ports	count(distinct id.resp_p)	Port scanning: many ports. brute-force: 1 port
mean_duration	mean(duration) over window	Bot beaconing: stable duration
std_duration	std(duration) over window	Bot beaconing: low variance (periodicity)
bytes_per_sec	sum(orig_bytes+resp_bytes)/window_duration	DDoS volumetric: high throughput

The sliding window is implemented as a per-source-IP data structure (IPWindow class) that maintains a list of recent flows. When a new flow arrives, entries older than 60 seconds are evicted, and the five aggregate statistics are recomputed from the remaining entries. The computational cost per flow is $O(k)$, where k is the number of flows in the window for the given source IP. Under normal traffic (k typically below 10), this overhead is negligible. Under high-rate flood attacks from a single IP, k can reach tens of thousands within the 60-second window, and the per-flow processing cost grows proportionally - this is a known scalability limitation of the current implementation. Per-IP isolation ensures that a high-rate attacker does not affect processing latency for other source IPs, but the attacker's own flows experience increased latency. An incremental $O(1)$ update using running counters would eliminate this bottleneck but is not implemented in the current prototype. Figure 3 illustrates the sliding-window mechanism: as the window advances, the five aggregate statistics are recomputed at each step on the remaining entries.

The temporal features are computed on per-day labeled.csv files processed in chronological order. The window state is reset between days to prevent information leakage across the dataset’s temporal boundaries. The resulting dataset contains 30,8 million flows with 17 features (12 baseline + 5 temporal) and is stored in Apache Parquet format.

Figure 3 - the sliding window approach:



Each source IP has its own independent window. Current prototype recomputes statistics over k entries in the window, resulting in $O(k)$ cost per flow.

2.8 Feature Engineering Design Decisions

Several design decisions require explicit discussion.

Why not CICFlowMeter’s 80 features? Beyond the implementation issues discussed in Section 1.2, CICFlowMeter presents a train-inference consistency problem for streaming deployment. Its feature vector includes full-flow aggregate statistics, packet inter-arrival-time metrics, active/idle time intervals, and packet length distributions. Reproducing these features during live inference would require additional packet-level buffering beyond Zeek conn.log and would make the inference pipeline different from the offline training pipeline. This work uses only Zeek conn.log fields to preserve consistency between offline training and streaming inference.

Why a 60-second window? The window size balances detection sensitivity against memory requirements. A shorter window (e.g., 10 seconds) would miss slower brute-force attacks with inter-attempt intervals of several seconds. A longer window (e.g., 300 seconds) would dilute the signal from short bursts and increase memory consumption per tracked IP. The 60-second value aligns with common rate-limiting thresholds in production firewalls and provides sufficient context for all attack types in the dataset.

Why not per-flow neural feature learning? Transformer-based approaches such as ET-BERT [19] and TrafficFormer [20] learn representations directly from packet byte sequences. These methods require GPU acceleration for real-time processing and operate on raw packets rather than structured conn.log records. Integrating them into a Kafka-based streaming pipeline where the feature extractor is Zeek would require a fundamentally different architecture. It can be addressed as future work.

Chapter 3: Model Training and Evaluation

3.1 Model Selection

Three gradient-boosted and ensemble classifiers are evaluated: Random Forest (RF), XGBoost, and LightGBM. The selection covers two fundamental algorithmic families - bagging (RF) and boosting (XGBoost, LightGBM) - and two practically distinct approaches to training on large-scale imbalanced data.

Random Forest builds an ensemble of decision trees via bagging with feature subsampling [12]. It serves as the interpretable baseline: feature importances are straightforward Gini impurity reductions, and the model is robust to irrelevant features without extensive hyperparameter tuning. On imbalanced data, `class_weight='balanced'` re-weights each class proportionally to its inverse frequency.

XGBoost [13] is a gradient-boosted tree implementation that uses an exact split-finding algorithm with L1/L2 regularization. It optimizes a second-order Taylor approximation of the loss function, enabling faster convergence than first-order methods. On imbalanced data, XGBoost requires explicit sample weights per row, computed as the inverse of class frequency.

LightGBM [14] uses a histogram-based split-finding approach that bins continuous features into discrete buckets before evaluating splits. This is substantially faster on datasets with tens of millions of rows while achieving comparable accuracy to XGBoost. LightGBM accepts `class_weight='balanced'` analogously to scikit-learn estimators.

CatBoost was considered but excluded from the final comparison: its ordered target statistics for categorical features require $O(n \times \text{cardinality})$ memory, and the history feature (approximately 5 000 unique values) causes out-of-memory errors on the full 30,8 million row dataset. Subsampling to 15 million rows would produce results not directly comparable to models trained on the complete data.

Neural network architectures (LSTM, GRU, attention-based models) were also excluded for three reasons:

1. Per-flow features in `conn.log` do not carry sequential structure within a single record - temporal patterns require the separate sliding-window aggregation described in Section 2.7.
2. Training recurrent networks on 30,8 million records requires GPU resources not available in the experimental setup.

3. Published surveys [16] report marginal F1 gains over gradient boosting on CICIDS datasets while incurring 10–100× higher training costs.

3.2 Experimental Setup

All experiments use the same 80/20 stratified random split (`random_state=42`) of the 30,8 million labeled flows, yielding approximately 24,6 million training samples and 6,2 million test samples. Stratification preserves the class distribution in both splits. The same preprocessing pipeline is applied to all models: $\log(1+x)$ transformation of seven numeric features, LabelEncoding of four categorical features, and integer representation of `id.resp_p`.

A time-based split (training on the first N days, testing on the remainder) was considered but not adopted. Attack class distribution is highly uneven across the ten capture days - DDoS-HOIC traffic is concentrated on February 21, the Bot class on March 2, the brute-force families on February 14 - and a chronological split would leave entire minority classes outside either the training or the test set, making per-class evaluation impossible. Stratified random split preserves class proportions in both halves. A stratified random split does not fully eliminate temporal dependence between flows from the same attack episode, however, it avoids the more severe problem of excluding entire minority classes from either training or testing. For this reason, the stratified random split is used for offline model comparison, while the live lab evaluation in Chapter 4 serves as an independent operational validation.

Each model is trained on all available CPU cores (Intel Xeon, 64 GB RAM). Hyperparameters are set to commonly used defaults: XGBoost (`n_estimators=200`, `max_depth=8`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`), Random Forest (`n_estimators=100`, `max_depth=20`), and LightGBM (`n_estimators=200`, `max_depth=8`, `learning_rate=0.1`). All models use inverse-frequency class weights.

Two evaluation metrics are reported. Weighted F1-score accounts for class imbalance by weighting each class by its support (number of test samples). Macro F1-score computes the unweighted average across all classes, giving equal importance to rare attack types regardless of their frequency. Macro F1 is the more informative metric for IDS evaluation because it reveals performance on minority attack classes that weighted F1 obscures.

Labeling noise sources: schedule-based labeling approach introduces several sources of label noise. All traffic from an attacker IP during the scheduled attack window is labeled as malicious, including background flows (DNS queries, ARP, TCP keepalives) that are

legitimate - this produces false positive labels in the training data. Note that IP addresses are not used as model features - the noise enters through labels, not inputs. If an attack extends beyond the scheduled window or starts with a delay, attack flows outside the window are labeled BENIGN, producing false negative labels. In CICIDS2018 the schedule is specified with minute-level granularity, while actual attack start and stop times may differ by tens of seconds. Clock drift between the Zeek capture timestamp and the attack schedule can further shift the labeling boundary. The approach also assumes a 1:1 mapping between IP addresses and roles. In networks with NAT this assumption breaks down, though CICIDS2018 uses direct internal addressing (172.31.x.x) without NAT. The net effect is unknown (small) but nonzero fraction of mislabeled flows in the training data, which ensemble tree models (Random Forest, XGBoost) tolerate well due to majority voting across trees, but which contributes to the gap between reported offline metrics and actual detection capability.

Despite these noise sources, the practical impact on offline metrics is limited. The large dataset size (30,8 million flows) averages out stochastic mislabeling during gradient boosting. Tree-based ensembles (Random Forest, XGBoost, LightGBM) are known to be robust to moderate label noise, since individual mislabeled examples cannot dominate split selection in deep trees, and the labelling is applied at attacker IP and time-window granularity rather than per-flow, which concentrates noisy labels in benign-looking traffic from attacker hosts (DNS, ARP, TCP keepalives) rather than distributing them randomly across the dataset. The live evaluation in Chapter 4, conducted in a controlled lab environment without schedule-based labelling ambiguity, provides an independent validation that offline-trained models generalize to clean ground truth.

3.3 Baseline Results (12 Features)

Table 6 presents the aggregate results for all three models trained on the 12-feature baseline dataset.

Table 6. Baseline model comparison (12 features, 30,8M flows)

Model	Weighted F1	Macro F1	Training time
Random Forest	0,9947	0,8687	770 s
XGBoost	0,9946	0,8613	1 418 s
LightGBM	0,9943	0,8588	377 s

All three models achieve weighted F1 above 0,994, which is expected given that the BENIGN class (89,1% of flows) is classified near-perfectly. The more revealing metric

is macro F1, where Random Forest leads at 0,8687. LightGBM trains $3.8\times$ faster than XGBoost with only a 0,25-percentage point difference in macro F1.

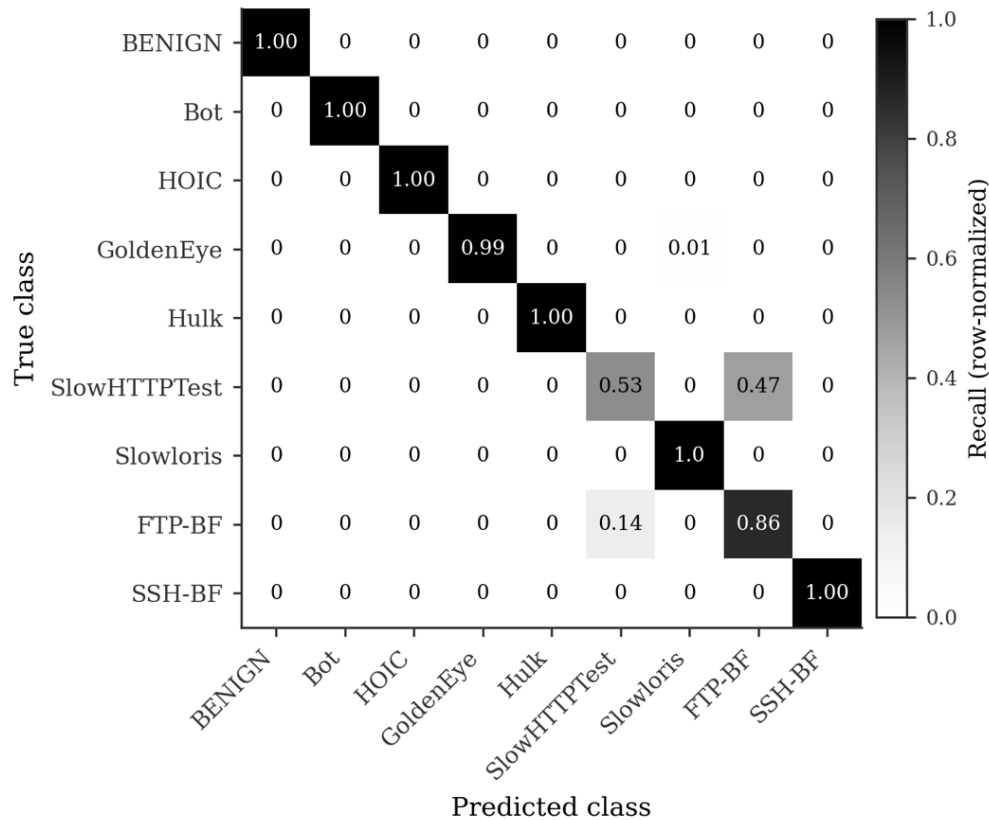
Table 7. Per-class F1-scores (baseline, 12 features)

Class	Random Forest	XGBoost	LightGBM
BENIGN	0,9980	0,9980	0,9980
DoS-Hulk	0,9999	0,9999	0,9999
DDOS-HOIC	0,9963	0,9962	0,9963
DoS-GoldenEye	0,9450	0,9422	0,9448
DoS-Slowloris	0,9657	0,9202	0,9608
SSH-BruteForce	0,9911	0,9912	0,9912
FTP-BruteForce	0,8179	0,8064	0,7674
DoS-SlowHTTPTest	0,6015	0,5948	0,5675
Bot	0,5032	0,5030	0,5030

Two classes present consistent difficulties across all models. Bot ($F1 \approx 0,5$) represents botnet command-and-control traffic from a Zeus/Ares campaign. Each individual conn.log record from a bot-infected host is indistinguishable from legitimate HTTP or IRC traffic: the duration, byte counts, and connection state fall within normal ranges. The attack signature is temporal - a bot host contacts the same external IP at regular intervals (beaconing), but per-flow features cannot capture this periodicity. The high-class weight increases recall but produces false positives from legitimate long-lived connections, resulting in low precision. This is a structural limitation of the per-flow approach, not a deficiency of any specific algorithm.

DoS-SlowHTTPTest ($F1 \approx 0,6$) sends partial HTTP requests that hold server connections open without transmitting data. In conn.log, these manifest as long-duration TCP connections with very small orig_bytes - similar to idle FTP control connections. The confusion between DoS-SlowHTTPTest and FTP-BruteForce observed in the per-class error analysis reflects the genuine similarity of their conn.log signatures at the per-flow level.

Figure 4 shows the confusion structure of the baseline XGBoost classifier:



Row-normalized confusion matrix for the baseline XGBoost classifier. The dominant off-diagonal pattern is the confusion between DoS-SlowHTTPTest and FTP-BruteForce: 47% of true DoS-SlowHTTPTest flows are classified as FTP-BruteForce, while 14% of true FTP-BruteForce flows are classified as DoS-SlowHTTPTest. This confirms that the 12 per-flow conn.log features cannot reliably separate long-duration, low-byte TCP sessions without temporal context. This error pattern motivates the temporal feature extension described in Section 3.4.

3.4 Temporal Results (17 Features)

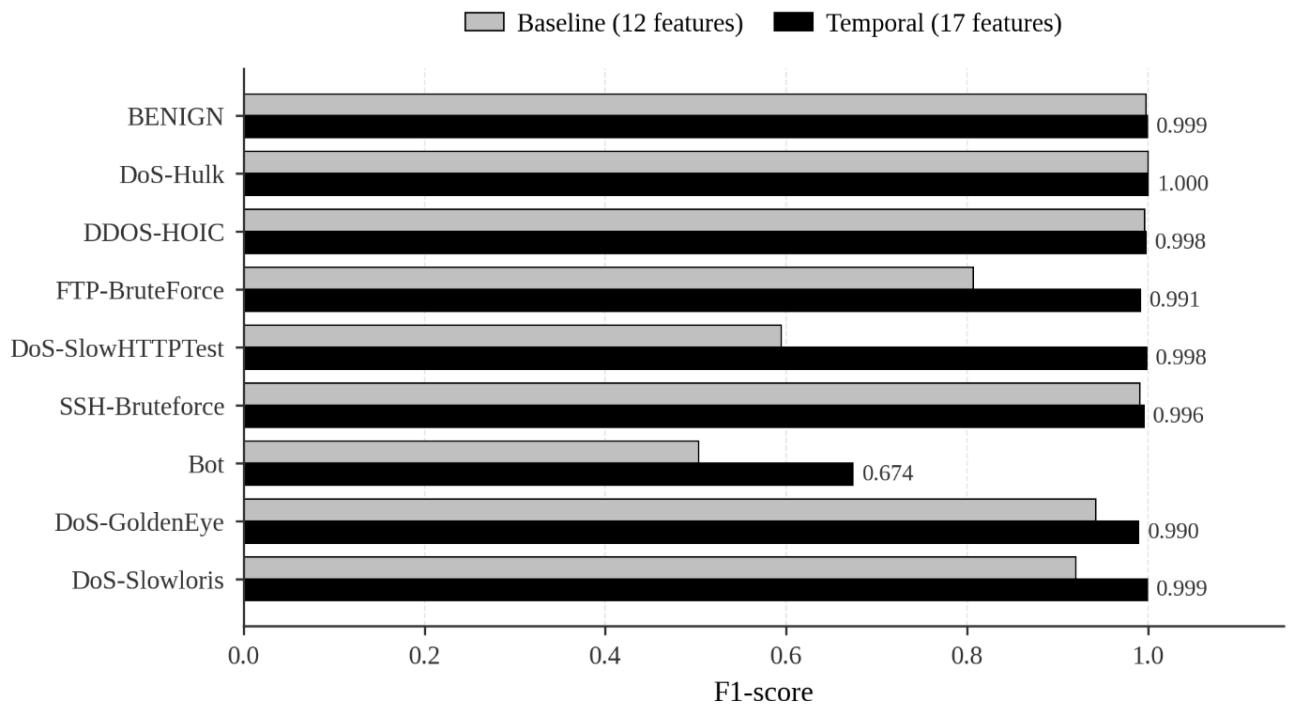
Table 8 presents the results after adding five temporal sliding-window features to the baseline feature set. All three models are retrained on the 17-feature temporal dataset using the same hyperparameters and train/test split.

Table 8. Temporal model comparison (17 features, 30,8M flows)

Model	Weighted F1	Macro F1	Δ Macro F1
Random Forest	0,9980	0,9528	+0,0841
XGBoost	0,9984	0,9605	+0,0991
LightGBM	0,9978	0,9465	+0,0877

Temporal features improve macro F1 by 8,4 to 9,9 percentage points across all models. XGBoost achieves the highest macro F1 of 0,9605 with temporal features - an improvement of +9,9 percentage points over its baseline. This model is selected for deployment in the real-time pipeline.

Figure 5 visualizes the per-class F1 improvement:



The largest improvements occurred in the classes that were most difficult for the baseline model. DoS-SlowHTTPTest improves from $F1 = 0,59$ to $F1 = 0,998$ - the temporal window captures the sustained low-throughput pattern that individual flows cannot represent. FTP-Bruteforce improves from $F1 = 0,81$ to $F1 = 0,99$, as the `conn_rate` feature directly encodes the rapid connection pattern characteristic of brute-force attacks. Bot improves from $F1 = 0,5$ to $F1 = 0,67$ - a substantial gain, though still the weakest class. The beaconing pattern is partially captured by `std_duration` (low variance from periodic connections), but the signal is weak because bot C2 intervals (minutes) are much longer than the 60-second window.

3.5 Autoencoder Experiment (Negative Result)

To evaluate whether unsupervised anomaly detection could complement the supervised classifier, we train an autoencoder on the 12-feature baseline dataset. The architecture consists of a symmetric encoder-decoder with layers $12 \rightarrow 8 \rightarrow 4 \rightarrow 8 \rightarrow 12$ using ReLU

activations. The model is trained exclusively on BENIGN flows (27,5 million samples) to learn a compressed representation of normal traffic. At inference time, flows with reconstruction error exceeding a threshold (set at the 99th percentile of training reconstruction errors) are flagged as anomalous.

The result is unambiguously negative: True Positive Rate = 0,01% (440 correctly flagged attack flows out of 3,35 million). At the 99th percentile threshold, the autoencoder correctly identifies only 440 of the 3,35 million attack flows in the test set, while producing approximately 275 000 false positives from BENIGN traffic.

The failure is structural rather than architectural. Per-flow conn.log features for most attack types fall within the statistical distribution of BENIGN traffic. A DoS-Hulk HTTP request produces a conn.log record with duration, byte counts, and packet counts indistinguishable from a legitimate HTTP request - the anomaly lies in the rate of connections from the same source, not in the properties of any individual connection. Similarly, an SSH brute-force attempt produces a normal-looking SSH connection record. The tested per-flow autoencoder cannot detect these attacks because the individual data points are not anomalous.

This negative result has two implications. First, it confirms that per-flow unsupervised detection is inadequate under the conn.log feature representation used in this work - a finding consistent with Sommer and Paxson's observation [17] that anomaly detection faces irreducible difficulties in network environments. Second, it motivates the temporal feature approach: if individual flows are indistinguishable, the detection signal must come from aggregate behavior over time. The temporal features introduced in Section 2.7 address exactly this limitation, and the +9,9 pp improvement in macro F1 confirms that the aggregate signal exists and is learnable. An autoencoder trained on temporal features rather than per-flow features remains a viable direction for future work on detection of previously unseen attack patterns.

3.6 Ensemble Methods Consideration

We briefly considered an ensemble combining XGBoost, Random Forest, and LightGBM via soft voting or stacking. The ensemble was not adopted for two reasons. First, weighted F1 already exceeds 0,994 for each individual model, the ceiling for improvement is negligible. Second, inference latency increases approximately $3\times$ (from 0,52 ms to 1,5 ms per flow) because all three models must be evaluated for each input. In a streaming pipeline processing 25 000–33 000 flows per second, this overhead reduces the throughput without meaningful accuracy gains.

3.7 Discussion

The offline evaluation reveals three key findings. First, all ensemble methods achieve near-identical weighted F1, confirming that the choice among RF, XGBoost, and LightGBM matters less than feature engineering for this task. Second, temporal features provide a dramatically larger improvement (+9,9 pp macro F1) than any algorithmic difference (< 1 pp between models). Third, per-flow conn.log features are insufficient for unsupervised anomaly detection in this experimental setting, as confirmed by the autoencoder's 0,01% TPR.

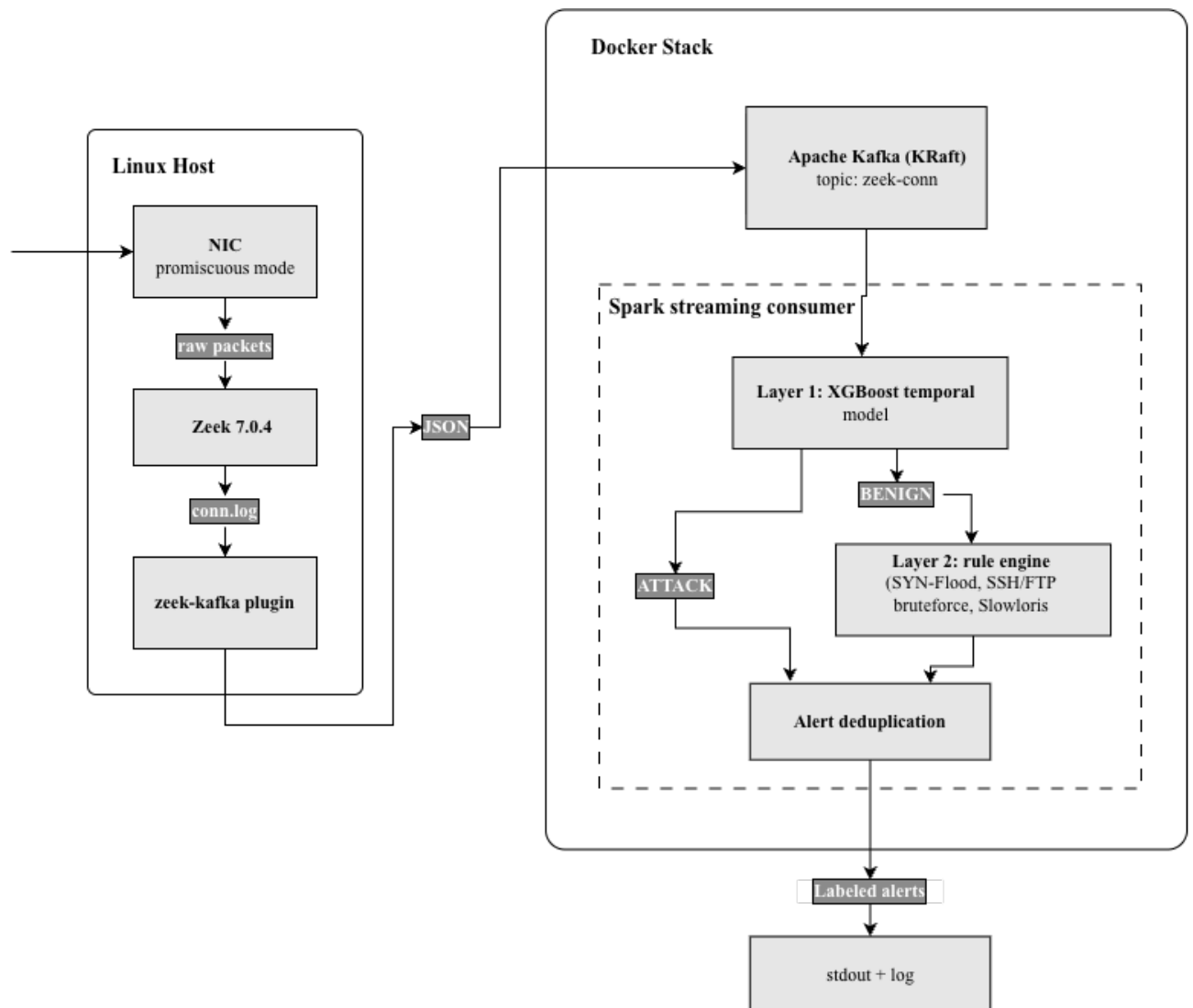
These offline metrics, however, do not predict live performance. A model with weighted F1 = 0,9946 might appear deployment-ready, but this metric is dominated by the 89% BENIGN class. The live testing described in Chapter 4 reveals that baseline models fail to detect 6 of 7 attack types despite their high offline scores - a finding that underscores the gap between benchmark evaluation and operational deployment.

Chapter 4: Real-Time System Design and Live Evaluation

4.1 Architecture Overview

The detection system follows a three-stage streaming architecture: capture, transport, and classification. Zeek monitors a network interface in promiscuous mode and produces a JSON-formatted conn.log record for each completed connection. The zeek-kafka plugin (v1.2.0, SeisoLLC) publishes these records to an Apache Kafka topic in real time. A Spark Structured Streaming consumer reads from the Kafka topic, extracts features, computes temporal sliding-window aggregates, applies the XGBoost classifier, evaluates rule-based detection logic, and emits alerts to the standard output.

Figure 6 - the architecture:



The separation of capture (Zeek), transport (Kafka), and classification (Spark consumer) allows each component to scale independently and fail without affecting the others. All three components run as Docker containers orchestrated by Docker Compose, requiring only a Linux server with Docker Engine and a network interface to monitor.

4.2 Kafka Streaming Transport

Apache Kafka operates in KRaft mode (no ZooKeeper dependency) with a single broker. The zeek-conn topic uses a single partition, which is sufficient for the throughput requirements of a single-server deployment. Zeek publishes conn.log records as JSON messages, the consumer deserializes them and extracts the 12 baseline features plus the source IP and timestamp needed for temporal window computation.

The choice of Kafka over direct IPC (e.g., Unix sockets) provides three benefits:

1. Durability - messages are persisted to disk, allowing the consumer to restart without data loss.
2. Decoupling - the consumer can fall behind the producer during traffic spikes and catch up during quieter periods.
3. Extensibility - additional consumers (e.g., a SIEM integration or an alerting service) can subscribe to the same topic independently.

4.3 Two-Layer Hybrid Detection

The consumer implements a two-layer hybrid detection architecture that combines the XGBoost temporal model with a stateful rule engine.

Layer 1: ML classification. For each micro-batch of flows (up to 10 000 per trigger interval), the consumer extracts the 12 baseline features, computes the 5 temporal features from the per-source-IP sliding window, encodes categorical features using the same LabelEncoder instances used during training, and applies the XGBoost model. The model produces a predicted class and a confidence score for each flow.

Layer 2: Rule engine. Flows classified as BENIGN by the ML model are further evaluated by the rule engine. This cascade design ensures that the rule engine acts as a safety net for attack types that the ML model misclassifies. Flows already classified as attacks by the ML model bypass the rule engine, avoiding duplicate alerts. Five rules are implemented:

Table 9. Rule engine specification

Rule	Trigger condition	Threshold	Window
SYN-Flood	conn_state in {S0, REJ}, port 80	≥ 50 conn/s, min 200 flows	60 s
SSH-BruteForce	port 22, any conn_state	≥ 0.5 conn/s, min 10 flows	60 s
FTP-BruteForce	port 21, any conn_state	≥ 0.5 conn/s, min 10 flows	60 s
Port-Scan	≥ 15 unique dst ports from same src	≥ 2 conn/s	60 s
Slowloris	duration ≥ 20 s, bytes ≤ 1000 , HTTP port	≥ 5 concurrent	60 s

The SYN-Flood rule counts both S0 (SYN with no reply) and REJ (SYN with RST reply) connection states, because the specific state depends on whether the target port is open or closed. The SSH and FTP brute-force rules count all connection states on the respective ports rather than filtering for failed connections only. This design decision reflects a fundamental limitation of conn.log: TCP connection state (S0, SF, REJ) does not indicate authentication success or failure, which occurs at the application layer. Zeek’s conn.log sees only the TCP handshake and data transfer, not the SSH or FTP login attempt within the encrypted session.

Rule thresholds were selected empirically during lab calibration and are intended as proof-of-concept values rather than production-ready defaults. Production deployment would require threshold tuning against local baseline traffic. All rules operate on per-source-IP sliding windows with a 60-second duration, matching the temporal feature window.

4.4 Consumer Implementation

The consumer is implemented in Python using Spark Structured Streaming for Kafka integration and micro-batch processing. Key implementation details include:

Feature extraction. JSON deserialization and feature extraction are vectorized using pandas operations (pd.to_numeric, dictionary-based categorical mapping) rather than row-by-row iteration. This optimization reduced per-batch latency from 9 000 ms to 496 ms ($18.1\times$ speedup).

Temporal window. The IPWindow class maintains a dictionary of per-source-IP deque structures. Each deque holds (timestamp, duration, orig_bytes, resp_bytes, resp_p) tuples. When a new flow arrives, expired entries (older than 60 seconds by wall-clock time) are evicted, and the five temporal statistics are recomputed from the remaining entries. A wall-clock guard clears stale windows that persist after an attack ceases - without this guard, temporal features computed from ghost entries would cause false positives after the attack ends.

Alert deduplication. Rather than emitting an alert for every attack flow, the consumer tracks active attack episodes per (source_ip, label) pair. A [NEW] alert is emitted when a source IP first appears with an attack classification. A [END] alert is emitted when no attack flows from that source are observed for a configurable timeout. The [END] alert includes summary statistics: total flows, average and maximum confidence, and wall-clock duration of the episode.

Model loading. The XGBoost model (xgb_baseline.json - filename retained from earlier development, the loaded artifact is the 17-feature temporal classifier described in Section 3.4) and LabelEncoder instances (label_encoder_xgb.joblib, label_encoder_cats_xgb.joblib) are loaded once at startup. The model and encoders are baked into the Docker image to eliminate runtime dependencies on external file systems.

4.5 Lab Testbed

Seven attack types were replayed in the live test environment. Six of them correspond directly to labels of the offline classifier (DoS-Hulk, DoS-GoldenEye, DoS-Slowloris, DoS-SlowHTTPTest, SSH-BruteForce, FTP-BruteForce). The seventh is a SYN flood, which is not a class in the training set and is therefore included as an unseen attack-type test of the temporal model's ability to generalise on traffic outside the training distribution. Two offline classes - Bot and DDoS-HOIC - were excluded from live replay: the original CIC-IDS2018 botnet captures rely on coordinated multi-host command-and-control behavior that cannot be reproduced faithfully in a two-host lab topology, and HOIC requires multiple coordinated booster hosts to reach its design throughput. The DDOS-HOIC label nevertheless appears in some live alerts as a cross-classification of DoS-Hulk traffic, since the two classes share the same volumetric HTTP-flood signature.

The seven attacks span three behavioural families: volumetric denial-of-service (SYN flood, Hulk, GoldenEye), slow-rate denial-of-service (Slowloris, SlowHTTPTest) and credential brute-force (SSH, FTP). Every attack was launched from the single Kali VM

against the lab victim, with runs of one to three minutes to keep the conn.log volume tractable.

Table 10. Attack specification and tools used

Attack	Tool and command	Behavioural family
SYN Flood	hping3 --flood	Volumetric (L4 TCP SYN)
DoS-Hulk	hulk_py2.py	Volumetric L7 HTTP
DoS-GoldenEye	goldeneye.py	Volumetric L7 HTTP
DoS-Slowloris	slowloris -s 500	Slow-rate L7 HTTP
DoS-SlowHTTPTest	slowhttptest -c 1000 -H -r 200 -u http://10.0.1.20 -t GET	Slow-rate L7 HTTP
SSH-BruteForce	hydra -t 16 fasttrack.txt (port 22)	Credential brute-force
FTP-BruteForce	hydra -t 16 fasttrack.txt (port 21)	Credential brute-force

Volumetric denial-of-service. **SYN Flood** was generated with `hping3 --flood`, producing a high rate of TCP SYN segments without completing the three-way handshake. Although SYN flood is not a class in the training set, the temporal classifier emitted a DoS-Hulk label on the resulting flows: the dominant feature signal - extreme `conn_rate` and `unique_dst_ports` - overlaps with Hulk's volumetric profile, and the model generalized within the same behavioral family. **DoS-Hulk** (HTTP Unbearable Load King, Python 2 implementation `hulk_py2.py`) issues a high rate of HTTP GET requests with randomized URI parameters and User-Agent strings to defeat caches and signature-based filters, the resulting traffic is characterized by short, high-bandwidth connections to TCP/80. **DoS-GoldenEye** (`goldeneye.py`) opens many parallel connections with KeepAlive and Cache-Control: no-cache headers, intended to exhaust the server's connection pool and bypass any caching layer. On the wire it appears as many concurrent long-lived HTTP exchanges.

Slow-rate denial-of-service. **DoS-Slowloris** (`slowloris -s 500`) opens 500 concurrent TCP connections to TCP/80 and sends one partial HTTP header line per connection every hundred seconds or so, holding the connections half-open indefinitely and exhausting thread-per-connection servers. **DoS-SlowHTTPTest** (`slowhttptest -c 1000 -H -r 200 -u http://10.0.1.20 -t GET`) extends the same idea with slow-read and slow-body variants and a higher target connection count. On the wire the two tools are indistinguishable at the `conn.log` granularity used by the consumer: both produce long-

lived connections to port 80 with a bytes-per-second rate close to zero. The rule engine therefore reports both under the Slowloris label - the live test is considered to have detected SlowHTTPTest if and only if the Slowloris rule fires during the SlowHTTPTest run. A practical observation: because Zeek writes a conn.log record only on connection termination, the SlowHTTPTest run is invisible to the consumer until the attacker tears down the sockets, at which point a burst of long-duration low-byte connections arrives in a single Spark micro-batch and triggers the rule.

Credential brute-force. SSH-BruteForce and FTP-BruteForce were generated with hydra -t 16 fasttrack.txt against TCP/22 and TCP/21 respectively, using the fasttrack wordlist and sixteen parallel threads. With sixteen parallel threads, the result observed in conn.log is a burst of TCP connections from one source to a single destination port, each connection carrying multiple credential attempts before the server closes it. The rule engine matches this pattern by counting connections per (src_ip, id.resp_p) pair within a sliding sixty-second window.

Live testing is conducted on a two-server lab testbed connected by a direct Ethernet link (1 Gbps).

Table 11. Lab testbed configuration

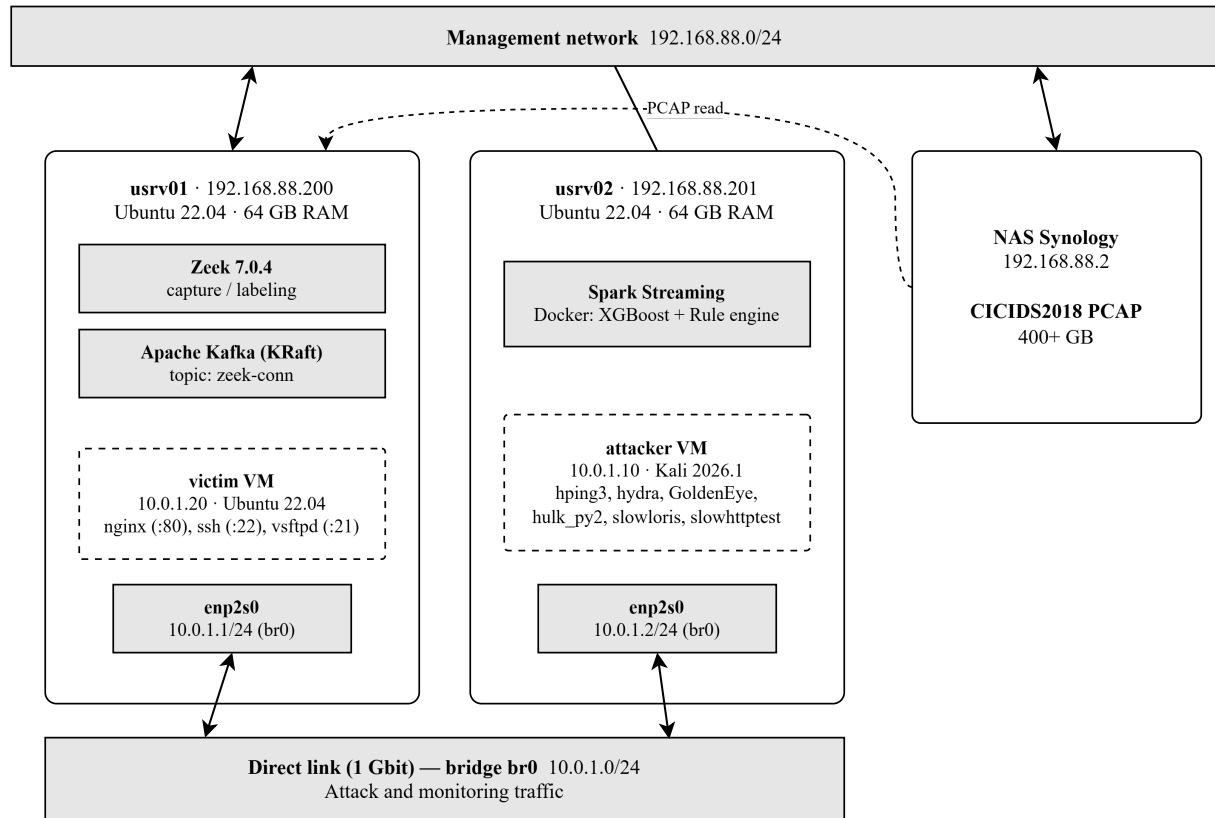
Host	Role	Hardware	OS
usrv01 (192.168.88.200)	Zeek, Kafka, model training	Intel Xeon, 64 GB RAM	Ubuntu 22.04
usrv02 (192.168.88.201)	Spark consumer, ML inference	Intel Xeon, 64 GB RAM	Ubuntu 22.04

Two KVM virtual machines provide the attack and victim roles:

VM	Host	IP	OS	Role
victim	usrv01	10.0.1.20	Ubuntu 22.04	nginx (port 80), SSH (port 22), vsftpd (port 21)
attacker	usrv02	10.0.1.10	Kali 2026.1	hydra, hping3, GoldenEye, slowloris, slowhttpstest

The servers are bridged via br0 (10.0.1.0/24). Zeek listens on br0 in promiscuous mode, capturing all traffic between the attacker and victim VMs. Management traffic on the 192.168.88.0/24 subnet (eno1 interface) is not monitored.

Figure 7 - the lab testbed setup:



4.6 Live Testing: Baseline ML Only

This section evaluates the baseline XGBoost model (12 features) without the rule engine or temporal features. The seven selected attack types described in Section 4.5 are exercised on the lab testbed.

Table 12. Live testing results - baseline ML only (12 features)

Attack	Tool	Result	Flows	Analysis
DoS-GoldenEye	GoldenEye.py	Detected	18 601	HTTP flood pattern, 93% detection rate
SYN Flood	hping3 –flood	BENIGN	1.5M+	L4 flood absent from CICIDS2018 training data
SSH-BruteForce	hydra	BENIGN	86	Conn.log blind to SSH auth, flows appear normal
Slowloris	slowloris.py	BENIGN	396	Zeek logs at connection close. attack invisible during execution
FTP-BruteForce	hydra	BENIGN	83	Same as SSH-BF: hydra reuses persistent FTP connections, auth events invisible at TCP level

DoS-Hulk	hulk_py2.py	BENIGN	204	Per-flow HTTP signature indistinguishable from normal traffic without temporal context
SlowHTTPTest	slowhttpstest	misclassified	1 161	Output mixed across Slowloris, GoldenEye and SYN-Flood labels, no consistent attack class assigned

The baseline model detects only 1 of 7 attack types in live conditions (DoS-GoldenEye, the single HTTP flood whose per-flow signature is sufficiently distinct). The remaining six failures reveal four structural limitations that motivate the hybrid architecture introduced in §4.7. First, SYN flood and DoS-Hulk are missed because their per-flow connections look ordinary in conn.log. The attack manifests only at the rate level, which the baseline 12-feature model cannot observe. Second, SSH and FTP brute-force are invisible at the conn.log level because authentication failures occur at the application layer (and inside an encrypted channel for SSH), leaving each flow indistinguishable from a normal SSH/FTP session. Third, Slowloris exploits a timing mismatch between the attack mechanism (holding connections open) and Zeek’s logging behavior (writing conn.log at connection close), so during execution the model sees nothing. Fourth, SlowHTTPTest produces flows that the baseline model oscillates between several attack labels (Slowloris, GoldenEye, SYN-Flood) without converging to a stable classification - the model recognizes anomaly but cannot assign a consistent class.

This result directly demonstrates the gap between offline metrics (weighted F1 = 0,9946) and live detection capability. The model performs well on the 89% BENIGN class but fails on multiple attack types that conn.log features cannot distinguish from legitimate traffic at the per-flow level.

4.7 Live Testing: Temporal Model with Rule Engine

The second round of live testing evaluates the full hybrid system: temporal XGBoost (17 features) with the rule engine enabled.

Table 13. Live testing results - temporal + rules (7 attack types)

Attack	Tool	ML Temporal	Rule Engine	Detection	Flows
DoS-Hulk	hulk_py2.py	Hulk, conf=1,00	-	Yes	125 864
DoS-GoldenEye	GoldenEye.py	GoldenEye, conf=0,90	-	Yes	19 000
SYN Flood	hping3 -flood	Hulk*, conf=0,95	-	Yes	1 210 616
SSH BruteForce	hydra	BENIGN	SSH-BF Rule	Yes	33

FTP BruteForce	hydra	BENIGN	FTP-BF Rule	Yes	67
Slowloris	slowloris.py	BENIGN	Slowloris Rule	Yes	396
SlowHTTPTest	slowhttpstest	BENIGN	Slowloris Rule	Yes	1 161

* Cross-class detection: model emitted a label from the same behavioral family but not the exact ground-truth class. SYN flood is not in the training label set and was generalized to *DoS-Hulk* because both share an extremely high conn_rate from a single source IP.

The hybrid system detects all 7 attack types tested, compared to 1 of 7 for the baseline ML model. The detection is split between two complementary layers: the ML temporal model handles volumetric DoS attacks where the temporal signal (high connection rate, many flows per source IP) is strong, while the rule engine handles attacks whose conn.log signatures are structurally indistinguishable from legitimate traffic at the ML feature level (SSH/FTP brute-force, Slowloris, SlowHTTPTest).

4.8 Key findings from live testing

Table 14. Contribution of each architectural component to overall detection performance:

Configuration	Features	Offline Macro F1	Live detection (lab)
Baseline ML	12	0,8613	1 / 7
+ Temporal features	17	0,9605 (+9.9 pp)	3 / 7
+ Rule engine	17	0,9605	7 / 7

Adding the five temporal sliding-window features to the 12-feature baseline improves offline macro F1 by 9,9 percentage points and substantially expands live coverage of the volumetric DoS family, including detection of an unseen SYN flood that the baseline classifier misses. Adding the rule engine on top of the temporal model lifts live detection to the full seven attacks without changing offline metrics, since the rules operate only on live conn.log records and are not part of the offline evaluation pipeline. The two layers are complementary rather than redundant: temporal ML carries the volumetric DoS family (DoS-Hulk, DoS-GoldenEye, SYN flood) where the statistical signal is strong, while the rule engine recovers the slow and brute-force families (Slowloris, SlowHTTPTest, SSH-BruteForce, FTP-BruteForce) where per-flow features blend with benign traffic but the duration / port / repetition pattern is unambiguous.

1. **Unseen-class generalization via temporal features.** SYN flood (hping3) is not represented in the CICIDS2018 training data. The temporal model nonetheless detects it with confidence 0,95, classifying it as DoS-Hulk. The connection rate from a single source IP executing a SYN flood is behaviorally similar to a DoS-Hulk HTTP flood, and the temporal features (conn_rate, bytes_per_sec) capture this similarity. This demonstrates that temporal features enable generalization to unseen attack types with similar behavioral patterns.
2. **Cross-classification between DoS variants.** GoldenEye produces flows, classified across multiple DoS categories (Hulk, GoldenEye, Slowloris) because HTTP flood tools generate traffic with overlapping temporal signatures. The attack is detected in all cases, but the specific label is imprecise. For operational purposes, the distinction between DoS subtypes is less important than the binary attack/benign decision.
3. **SSH/FTP brute-force is visible only indirectly in conn.log.** The authentication failures themselves occur at the application layer and are not present in conn.log. Detection in this work therefore relies on cross-flow aggregates: the rule engine counts repeated connections from the same source IP to TCP/22 or TCP/21 within a 60-second window, and the temporal ML features (conn_rate, unique_dst_ports) capture the same rate-based signal. The rule engine remains useful as an explicit, interpretable layer for these attacks, extending the pipeline to consume ssh.log and ftp.log for direct application-layer detection is a natural direction for future work.
4. **Slow attacks exploit Zeek's logging timing.** Slowloris and SlowHTTPTest hold HTTP connections open indefinitely, sending minimal data. Zeek writes conn.log only when a connection closes. While the attack is active, the temporal window contains no entries for the slow connections, and the ML model sees no signal. The rule engine detects these attacks by evaluating individual flow properties (long duration + low bytes + HTTP port) at the moment the conn.log record is written. This represents a fundamental trade-off of flow-based detection: the system cannot classify a connection until it is complete.
5. **Offline metrics overestimate live performance.** The baseline model achieves weighted $F1 = 0,9946$ offline but detects only 1 of 7 attack types live (14% detection rate). Even the temporal model, with macro $F1 = 0,9605$ offline, fails on 4 of 7 attack types that require rule-based detection. This gap arises because offline evaluation benefits from completed flows and full temporal context,

whereas live detection operates on a continuously evolving stream where slow attacks may not produce conn.log records until after they end.

4.9 Docker Deployment

The complete system is containerized using Docker Compose with three services: Kafka (KRaft mode, official Apache image), Zeek (custom image: zeek/zeek:7.0.4 + zeek-kafka plugin), and the Spark consumer (custom image: spark:python3 + XGBoost + trained model). All containers use host networking to allow Zeek direct access to the monitoring interface.

Smoke testing of the Docker deployment confirms detection of all seven tested attack types.

Table 15. Docker deployment smoke test results:

#	Attack (run)	Tool / command	Detected as	Layer	Total flows	Conf avg / max
1	SYN Flood (unseen class)	hping3 --flood	DoS-Hulk*	ML	1 214 031	0.95 / 1.00
2	DoS-Hulk	hulk_py2.py	DoS-Hulk + DDOS-HOIC*	ML	173 750	1.00 / 1.00
3	DoS-GoldenEye	goldeneye.py	DoS-GoldenEye + DoS-Hulk*	ML	23 906	0.94 / 1.00
4	DoS-Slowloris	slowloris -s 500	Slowloris	Rule	28 298	0.95 / 0.95
5	SSH-BruteForce	hydra -t 16 fasttrack.txt	SSH-BruteForce	Rule	32	0.95 / 0.95
6	FTP-BruteForce	hydra -t 16 fasttrack.txt	FTP-BruteForce	Rule	67	0.95 / 0.95
7	DoS-SlowHTTPTest	slowhttpptest -c 1000 -H -r 200	Slowloris*	Rule	761	0.95 / 0.95

* Cross-class detection: model emitted a label from the same behavioural family but not the exact ground-truth class. SYN flood is not in the training label set and was generalised to *DoS-Hulk* because both share an extremely high conn_rate from a single source IP. Hulk and HOIC alternate during volumetric HTTP runs because they share the same flow-level statistical signature. SlowHTTPTest is reported under the *Slowloris*

label because both tools produce indistinguishable conn.log signatures (long-lived port-80 connections with near-zero throughput).

Table 15 reports the per-attack results of the Docker smoke test. All seven attack types were detected, matching the bare-metal results in §4.7. The ML temporal model handled the three volumetric runs - SYN flood, Hulk and GoldenEye - with the SYN flood, absent from the training label set, generalizing to a DoS-Hulk cross-class classification (run 1, confidence 0,95). Cross-classification between DoS-Hulk and DDOS-HOIC for the volumetric HTTP runs reflects the genuine similarity of their conn.log signatures: both tools generate the same statistical pattern at the flow level, and the model alternates between the two labels within a single attack burst. The four slow and brute-force runs (Slowloris, SlowHTTPTest, SSH-BruteForce, FTP-BruteForce) were detected by the rule engine. ML model classified these flows as BENIGN because their per-flow features remain within the benign distribution. SlowHTTPTest is reported under the Slowloris rule label because two tools produce indistinguishable conn.log signatures - long-lived port-80 connections with bytes-per-second close to zero.

4.10 Performance

Table 16. Throughput comparison:

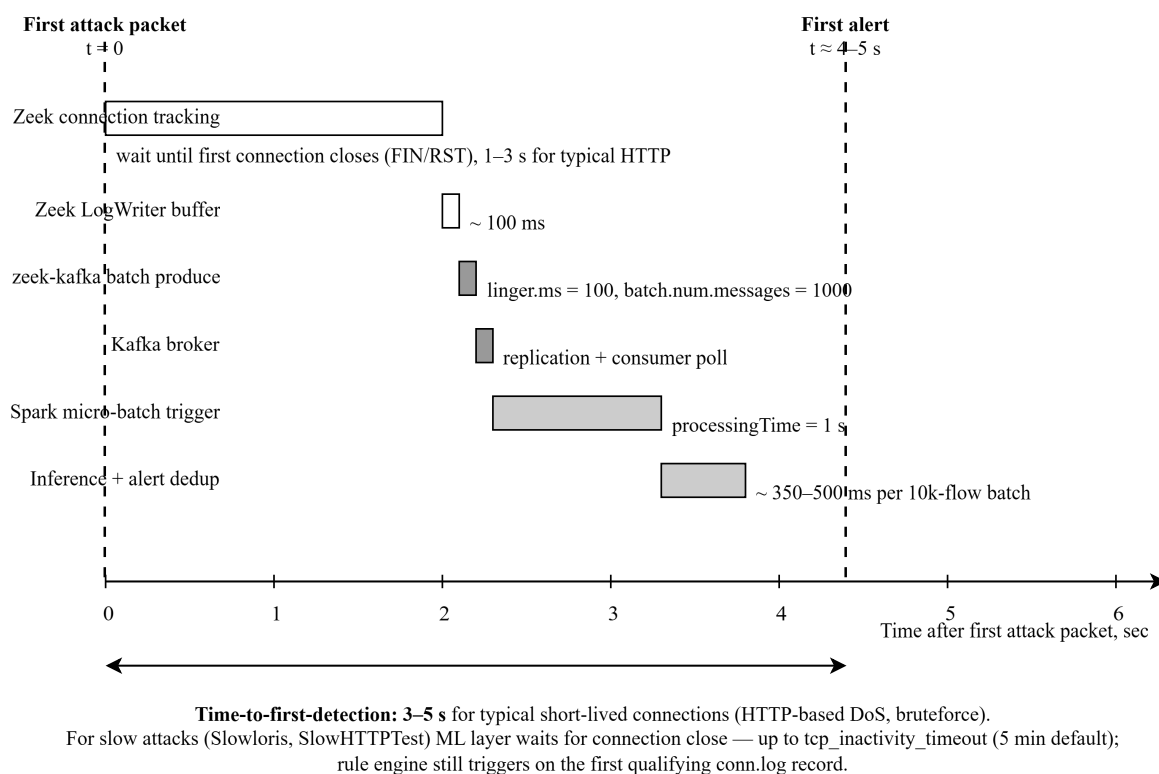
Deployment	Throughput (flows/sec)	Temporal overhead per batch
Bare metal (usrv01 + usrv02)	25 000–33 000	20–80 ms
Docker (single server, usrv01)	24 000–30 000	260–390 ms

The bare-metal deployment sustains 25 000–33 000 flows per second with temporal feature computation adding 20–80 ms of overhead per micro-batch. The Docker deployment achieves comparable per-batch throughput to bare metal (24 000–30 000 flows per second during peak DoS-Hulk batches, with single-batch peaks reaching 30 450 flows per second), but with higher per-batch temporal feature overhead (260–390 ms versus 20–80 ms on bare metal). The increased overhead reflects shared CPU resources on a single server and container scheduling, not the throughput ceiling itself. Both configurations exceed the traffic volume of the lab testbed (peak ~5,000 flows/sec during DoS attacks).

Detection latency approximation - the time between the first packet of an attack and the first alert - is 3–5 seconds for typical short-lived connections (HTTP-based DoS variants, FTP/SSH brute-force). The cumulative pipeline overhead consists of Zeek connection tracking until the first qualifying connection closes via FIN/RST (1–3 seconds for typical HTTP traffic, the dominant component). Zeek LogWriter buffering (~100 ms).

zeek-kafka batch production with `linger.ms = 100` and `batch.num.messages = 1000`. Kafka broker hop and Spark micro-batch trigger of up to one second (`processingTime = 1 s`). Model inference together with alert deduplication ($\sim 350\text{--}500$ ms per 10 000-flow batch).

Figure 8 - latency breakdown:

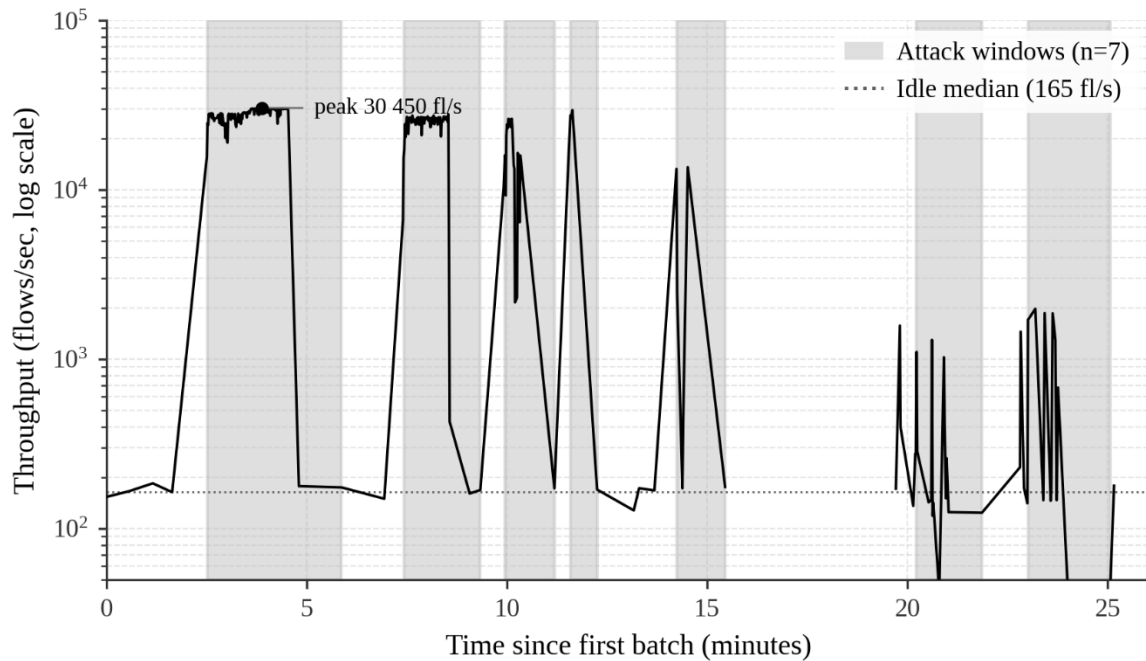


Slow attacks that hold connections open (Slowloris, SlowHTTPTest) cannot be detected by the ML layer until the connection terminates, since Zeek writes a `conn.log` record only on closure - for these attacks ML detection waits up to the value of `tcp_inactivity_timeout` (5 minutes by default in Zeek). The rule engine, however, evaluates each `conn.log` record individually and triggers on the first qualifying flow, providing detection at the moment the record is written regardless of how long the connection itself was open.

The pipeline also exhibits a 15–20 second tail between the last packet of an attack and the final flow arriving at the consumer, reflecting the same buffering and trigger components plus residual connection-tracking timeouts. This tail influences the duration field of the [END] alert but does not affect time-to-first detection. Packet-based IDS

(e.g., Suricata) detect the same attacks with lower latency but without access to flow-level statistical features.

Figure 9 – throughput of the Spark consumer during the live smoke test:



Each point is one Spark micro-batch, the line shows the per-batch flow rate (log scale). Gray bands mark the seven attack windows reconstructed from [SPAN] aggregation events emitted by the consumer (per-source wall_clock duration). The idle median is 165 flows/sec, the peak of 30 450 flows/sec occurred during the DoS-Hulk volumetric flood. Volumetric attacks (DoS-Hulk, DDoS-HOIC, DoS-GoldenEye) appear as one to two orders of magnitude surges above idle, while the rule-based brute-force detections (SSH, FTP) operate at near-idle volume - these attacks are caught by behavioral rules rather than by traffic volume. The interval 15,5-19,5 min is omitted (idle gap between attack windows).

4.11 Limitations

The live evaluation reveals several limitations of the current system:

Single attacker IP. All tests use a single attacker IP (10.0.1.10), which means temporal window state from one test may contaminate subsequent tests if not cleared. In production, multiple distinct attackers would not share temporal state.

Cross-classification between similar DoS types. The ML model assigns imprecise labels to HTTP-based DoS attacks because their temporal signatures overlap. The binary attack/benign decision is correct, but the specific attack category may not be.

SSH/FTP brute-force detection is heuristic. The rule engine counts connections per port without access to authentication results. Legitimate SSH activity from a single IP (e.g., automated deployments) could trigger false positives if it exceeds the threshold.

Cascade logic limitation. When the ML model classifies a flow as an attack, the rule engine is bypassed for that flow. This means the SYN-Flood rule never fires when the temporal model already classifies SYN flood flows as DoS-Hulk.

Lab-scale evaluation only. The testbed uses two servers and a single attacker. Production networks with thousands of concurrent sources, encrypted traffic, and mixed legitimate/attack traffic would present additional challenges not evaluated here.

Threshold calibration. Rule thresholds were selected empirically on the lab testbed and were not calibrated against production-like benign background traffic. Production deployment would require tuning each threshold against a local baseline to control the false positive rate.

Temporal aggregation complexity. The current sliding-window implementation recomputes statistics over a per-source-IP queue with $O(k)$ cost per flow, where k is the number of flows from that source within the last 60 seconds. Under concentrated high rate floods this cost can become non-trivial. Production deployment should replace the re-computation with $O(1)$ running counters as discussed in 1.6.

False positive rate not measured under realistic background. Live tests were conducted against isolated lab traffic rather than mixed production-like benign traffic. The detection rate of 7 of 7 attack types therefore measures sensitivity, but the false positive rate of the rule engine and ML classifier on real-world background load is outside the scope of this evaluation.

Conclusion

This thesis presented an end-to-end flow-level near real-time network intrusion detection system that combines supervised machine learning with rule-based detection, deployed as a containerized streaming pipeline.

The work addressed four research gaps identified in the literature review. The first gap - feature reliability - was addressed by processing the CSE-CIC-IDS2018 dataset (400+ GB of raw PCAP, 30,8 million flows) exclusively through Zeek conn.log, bypassing the CICFlowMeter tool and its documented implementation bugs. The second gap - train-inference consistency - was addressed by using the same feature extractor (Zeek with zeek-kafka plugin) in both the offline training pipeline and the real-time inference pipeline, eliminating the distribution shift that arises when training and inference use different tools. The third gap - end-to-end deployment architecture - was addressed by implementing and testing a complete streaming pipeline from raw network packets to classification alerts, containerized with Docker Compose for reproducible deployment. The fourth gap - temporal features for flow-level IDS - was addressed by introducing five sliding-window features computed per source IP over a 60-second window, which improve XGBoost macro F1.

The key results are as follows. Three ensemble classifiers (Random Forest, XGBoost, LightGBM) trained on 12 baseline Zeek conn.log features all achieve weighted F1 above 0,994 on the offline test set. However, live testing reveals that baseline models detect only 1 of 7 tested attack types (14% detection rate), demonstrating a severe gap between offline benchmark metrics and operational performance.

Temporal sliding-window features - five aggregate statistics computed per source IP over a 60-second window - improve macro F1 from 0,8613 to 0,9605 (+9,9 percentage points) for XGBoost, with the largest gains in DoS-SlowHTTPTest (+40 pp), FTP-BruteForce (+18 pp), and Bot (+17 pp). The temporal model also demonstrates the ability to detect an attack type absent from training: SYN flood, absent from the training data, is detected with confidence 0,95 because its temporal signature (extremely high connection rate from a single source) matches the DoS-Hulk pattern.

The two-layer hybrid architecture (temporal ML + rule engine) detects all 7 tested attack types in the controlled lab testbed, compared to 1 of 7 for baseline ML alone. The rule engine compensates for structural limitations of flow-level ML: SSH and FTP authentication failures are invisible to conn.log, because authentication occurs at the application layer, and slow attacks (Slowloris, SlowHTTPTest) exploit Zeek's connection-closure logging timing.

A per-flow autoencoder anomaly detector achieves only $\text{TPR} = 0,01\%$, confirming that per-flow features lack the contextual signal needed for unsupervised separation on conn.log data. This negative result validates the supervised + rule-based approach adopted in this work.

The system sustains 25 000 - 33 000 flows per second on commodity hardware in bare-metal deployment and reaches 24 000 - 30 000 flows per second during peak Docker batches. Detection latency is 3 - 5 seconds for time to first alert for short-lived attacks, with a 15 - 20 second tail driven by Zeek connection close logging behavior.

Several limitations constrain the generalizability of these results. The evaluation is conducted on a two-server lab testbed with a single attacker IP, which does not reproduce the scale and diversity of production network traffic. Cross-classification between similar DoS variants produces correct binary (attack/benign) decisions but imprecise attack category labels. SSH and FTP brute-force detection relies on connection-rate heuristics rather than application-layer authentication data.

Future work should proceed in three directions:

First, extending the pipeline to consume Zeek's application-layer logs (ssh.log, http.log) would provide direct access to authentication success/failure events, replacing the current heuristic rules with precise detection for brute-force attacks.

Second, applying unsupervised anomaly detection to temporal-aggregated features rather than per-flow features may enable detection of previously unseen attack patterns without labeled training data.

Third, a distributed deployment across multiple servers with horizontal scaling of Kafka partitions and Spark executors would allow evaluation at production-relevant traffic volumes. Such distributed scaling was kept outside the scope of the present prototype.

References

- [1] Getman A.I., Goryunov M.N., Matskevich A.G., Rybolovlev D.A. Methodology for Collecting a Training Dataset for an Intrusion Detection Model. *Trudy ISP RAN/Proc. ISP RAS*, vol. 33, issue 5, 2021, pp. 83–104. DOI: 10.15514/ISPRAS-2021-33(5)-5
- [2] Anderson J.P. *Computer Security Threat Monitoring and Surveillance*. James P. Anderson & Co., Fort Washington, PA, 1980.
- [3] Denning D.E. An Intrusion-Detection Model. *IEEE Transactions on Software Engineering*, 13(2):222–232, 1987.
- [4] Roesch M. Snort - Lightweight Intrusion Detection for Networks. *Proceedings of USENIX LISA*, 1999.
- [5] Open Information Security Foundation. Suricata Open Source IDS/IPS/NSM Engine. URL: <https://suricata.io>.
- [6] Stolfo S., Fan W., Lee W., Prodromidis A., Chan P. KDD Cup 1999 Data. UCI Machine Learning Repository, 1999.
- [7] Tavallae M., Bagheri E., Lu W., Ghorbani A. A Detailed Analysis of the KDD CUP 99 Data Set. *Proc. IEEE CISDA*, 2009.
- [8] Moustafa N., Slay J. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. *Proc. MilCIS*, 2015.
- [9] Sharafaldin I., Habibi Lashkari A., Ghorbani A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proc. ICISSP*, 2018.
- [10] Sharafaldin I., Habibi Lashkari A., Ghorbani A. CSE-CIC-IDS2018 on AWS. Canadian Institute for Cybersecurity, 2018. URL: <https://www.unb.ca/cic/datasets/ids-2018.html>.
- [11] G. Engelen, V. Rimmer and W. Joosen, "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study," 2021 IEEE Security and Privacy Workshops (SPW), San Francisco, CA, USA, 2021, pp. 7-12, doi: 10.1109/SPW53761.2021.00009.
- [12] Breiman L. Random Forests. *Machine Learning*, 45(1):5–32, 2001.
- [13] Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System. *Proc. ACM SIGKDD*, pp. 785–794, 2016.
- [14] Ke G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in NeurIPS*, 2017.

- [15] Leevy J.L., Khoshgoftaar T.M. A Survey and Analysis of Intrusion Detection Models Based on CSE-CIC-IDS2018 Big Data. *Journal of Big Data*, 7, Article 104, 2020. DOI: 10.1186/s40537-020-00382-x.
- [16] Ferrag M.A., Maglaras L., Moschoyiannis S., Janicke H. Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study. *Journal of Information Security and Applications*, 50:102419, 2020. DOI: 10.1016/j.jisa.2019.102419.
- [17] Sommer R., Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *Proc. IEEE S&P*, 2010.
- [18] Gustavsson V. Machine Learning for a Network-based Intrusion Detection System: An application using Zeek and the CICIDS2017 dataset. BSc Thesis, KTH Royal Institute of Technology, 2019.
- [19] Lin X. et al. ET-BERT: A Contextualized Datagram Representation with Pre-Training Transformers for Encrypted Traffic Classification. *Proc. ACM Web Conference (WWW)*, 2022. DOI: 10.1145/3485447.3512217.
- [20] Zhou G., Guo X., Liu Z., Li T., Li Q., Xu K. TrafficFormer: An Efficient Pre-trained Model for Traffic Data. *IEEE Symposium on Security and Privacy (SP)*, 2025.
- [21] Kreps J., Narkhede N., Rao J. Kafka: A Distributed Messaging System for Log Processing. *Proc. NetDB*, 2011.
- [22] Carbone P. et al. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin*, 38(4):28–38, 2015.
- [23] Zaharia M. et al. Discretized Streams: Fault-Tolerant Streaming Computation at Scale. *Proc. SOSP*, 2013.
- [24] Ioannou C., Vassiliou V. Network Attack Classification in IoT Using Support Vector Machines. *Journal of Sensor and Actuator Networks*, 10(3):58, 2021. DOI: 10.3390/jsan10030058
- [25] Ahmad I. et al. Performance Comparison of Support Vector Machine, Random Forest, and Extreme Learning Machine for Intrusion Detection. *IEEE Access*, 6:33789–33795, 2018.
- [26] Zubkov A.V. Development of a Microservice Architecture for Streaming Network Traffic Events [Razrabotka mikroservisnoy arkhitektury potokovoy peredachi sobytiy setevogo trafika]. Final project report. National Research University Higher School of

Economics, Centre for Continuing Education, Faculty of Computer Science, Moscow, 2022.

[27] Gnanavarothayan K., Moelchand P., van Stam J., Verheijde J. Real Time Threat Detection Through Network Analysis. Bachelor's thesis. Delft University of Technology, Rotterdam, 2019. DOI: 10.13140/RG.2.2.32802.17607.

[28] El Mahdaouy A., Ait Yahia I., Oualil S., Berrada I. Deep Learning for Contextualized NetFlow-Based Network Intrusion Detection: Methods, Data, Evaluation and Deployment. arXiv:2602.05594, 2026.

[29] Mo J., Ke J., Zhou H. et al. Hybrid Network Intrusion Detection System Based on Sliding Window and Information Entropy in Imbalanced Dataset. Applied Intelligence, Springer, 2025. DOI: 10.1007/s10489-025-06307-6

[30] Raskovalov A., Gabdullin N., Androsov I. NIDS Neural Networks Using Sliding Time Window Data Processing with Trainable Activations and its Generalization Capability. arXiv:2410.18658, 2024.

[31] Zhong M., Lin M., Zhang C., Xu Z. A Survey on Graph Neural Networks for Intrusion Detection Systems: Methods, Trends and Challenges. Computers & Security, 141:103821, 2024. DOI: 10.1016/j.cose.2024.103821

[32] fisher85. Я больше не верю публичным датасетам [I no longer trust public datasets]. Habr, ИСП РАН, 13 августа 2024. URL: https://habr.com/ru/companies/isp_ras/articles/829506/.